

Mehran Rafiee

INTELLIGENT BIM ASSISTANT BY LEVERAGING LARGE LANGUAGE MODELS

Master's thesis
Faculty of Information Technology and Communication Sciences
July 2025

ABSTRACT

Mehran Rafiee

INTELLIGENT BIM ASSISTANT BY LEVERAGING LARGE LANGUAGE MODELS

Master's thesis

Tampere University

Computing Sciences - Machine learning and signal processing

July 2025

This thesis focuses on developing an AI-powered tool that simplifies the exploration of Building Information Modeling (BIM) data for Solibri users by leveraging the Neo4j graph database and OpenAI's large language model (LLM). The system translates natural language prompts into meaningful Cypher queries executed on the Neo4j database, enabling advanced search and validation capabilities without requiring users to have expert knowledge of filters and rules. The project encompasses comprehensive data conversion and cleaning, thoughtful database architecture and modeling, and the creation of a robust data insertion pipeline. The end result is an AI agent that generates accurate Cypher queries from human prompts, streamlining BIM workflows through efficient element discovery and compliance verification. This work underscores the potential of LLMs to transform BIM applications by reducing manual workload and allowing professionals to focus on more complex, human-centric tasks.

Keywords: BIM, LLM, IFC, graph database, AI, AI Agent

The originality of this thesis has been checked using the Turnitin Originality service.

PREFACE

This thesis was conducted as part of a company project at Solibri Oy and completed in partial fulfillment of the requirements for the degree at Tampere University. The work explores the use of large language models to build intelligent assistance capabilities in the context of Building Information Modeling (BIM).

I would like to express my sincere gratitude to my supervisor, Professor Jyrki Nummenmaa, for his guidance and constructive feedback throughout the process. His support has been valuable in shaping both the direction and quality of this thesis.

The collaboration with Solibri Oy provided meaningful insights and practical grounding for the research, and I am thankful for the opportunity to work on a topic that bridges academic research with real-world applications.

Tampere, 30th July 2025

Mehran Rafiee

CONTENTS

1	INTRODUCTION	1
1.1	BACKGROUND	1
1.2	BUILDING INFORMATION MODELS	2
1.3	BIM SEMANTICS	2
1.4	OBJECTIVE	4
1.5	RESEARCH SCOPE AND METHODOLOGY	5
1.6	SIGNIFICANCE OF THIS RESEARCH	6
2	KEY TECHNOLOGIES	7
2.1	GRAPH AND GRAPH DATABASE	7
2.2	SERVERLESS COMPUTING	10
2.3	LARGE LANGUAGE MODELS (LLM)	11
2.4	INDUSTRY FOUNDATION CLASSES (IFC)	12
2.5	JSON AND GLTF FORMATS	13
3	PAST ACHEIVEMENTS	15
3.1	SOLIBRI DESKTOP	15
3.2	DATA EXPORTER	15
3.3	PREVIOUS AI APPROACHES IN BIM	16
4	METHODOLOGY AND TECHNICAL APPROACH	18
4.1	SYSTEM ARCHITECTURE DESIGN	18
4.2	SELECTING A SUITABLE DATABASE	20
4.2.1	RELATIONAL DATABASES	21
4.2.2	DOCUMENT DATABASES	22
4.2.3	GRAPH DATABASES	22
4.2.4	REASONING AND CONCLUSION	23
4.3	IMPLEMENTATION	28
4.3.1	SMC FILE EXTRACTION	28

4.3.2	LAMBDA FUNCTIONS	28
4.3.3	DATA TRANSFORMATION AND IMPORTING TO DATABASE	29
4.3.4	AI AGENT	29
4.3.5	OUTPUT STRUCTURE	30
5	TESTING AND VERIFICATION	34
5.1	TESTING APPROACH	34
5.2	EVALUATION AND BENCHMARKING	36
5.3	LIMITATIONS	37
6	FUTURE WORK	40
7	CONCLUSION	42
	REFERENCES	43

USE OF ARTIFICIAL INTELLIGENCE IN THIS WORK

Artificial intelligence (AI) has been used in generating this work:

- ☒ Yes
☐ No

I hereby declare, that the AI-based applications used in generating this work are as follows:

Application	Version
OpenAI ChatGPT	GPT-4
Grammarly	latest 2025 July

PURPOSE OF THE USE OF AI

AI was used to assist in writing, refining, and validating the content of this thesis. ChatGPT (GPT-4) was used to help formulate research questions, clarify technical descriptions, generate example code (in Python and Cypher), and suggest logical ways to structure sections. It also assisted in rephrasing sentences to improve clarity and coherence. Grammarly was used to correct grammar, punctuation, and style.

AI was not used to generate entire sections autonomously; all outputs were reviewed, edited, and integrated by the author.

PARTS OF THIS WORK, WHERE AI WAS USED

Chapter 1: Assistance in structuring and phrasing the introduction and research questions.

Chapter 2: Summarization and rewording of key technologies content.

Chapter 4: Formulation of methodology description structures for thesis writing.

Chapter 5: Generation of Python script foundations and Cypher queries for semi-automated testing.

In addition to these, AI engines were used to perform spell checking, grammar corrections, and overall coherence of the chapters.

ACKNOWLEDGEMENT OF RISKS

I hereby acknowledge, that as the author of this work, I am fully responsible for the contents presented in this thesis. This includes the parts that were generated

by an AI, in part or in their entirety. I therefore also acknowledge my responsibility in the case, where use of AI has resulted in ethical guidelines being breached.

1 INTRODUCTION

1.1 BACKGROUND

Building Information Modeling (BIM) has fundamentally transformed the Architecture, Engineering, and Construction (AEC) industry by providing a digital, structured representation of buildings and infrastructure throughout their lifecycle. BIM enables stakeholders to collaboratively design, analyze, and manage built assets using rich datasets that combine geometric and semantic information. These structured representations typically consist of two key components: geometry, which captures the physical form and spatial relationships of building elements, and semantics, which define the meaning, classification, and properties of those elements.

Software platforms such as Solibri have been instrumental in realizing the full potential of BIM in practice. Solibri allows users to perform model validation, rule-based checking, and quality assurance tasks in a single environment. The software offers powerful tools for running highly configurable checks to ensure that models comply with project-specific requirements, industry standards, and regulatory codes. By identifying design issues before construction begins, Solibri helps reduce rework, minimize costly errors, and streamline project delivery. Additionally, its visualization features and element-filtering capabilities enable users to conduct detailed inspections and navigate large models with ease.

Despite its advanced capabilities, configuring and using Solibri efficiently often requires a steep learning curve. Expert users must manage complex rule sets and filters, while less experienced users may struggle to perform even basic operations without training. The interface, while comprehensive, assumes a certain level of domain knowledge and familiarity with BIM data structures. As a result, there remains a gap between the software's capabilities and its accessibility for a broader range of users, including architects, engineers, and project managers who may not be specialists in BIM tools or data querying.

Recent advancements in artificial intelligence, particularly in the field of Large Language Models (LLMs), have introduced promising new avenues for enhancing software usability. LLMs such as OpenAI's GPT series have demonstrated impressive capabilities in understanding and generating natural language, making them ideal candidates for bridging the gap between human users and technical systems. Through natural language processing (NLP), LLMs can interpret user intent and translate it into machine-executable queries or actions. This enables more intuitive interactions with software tools and reduces the need for specialized training.

In the context of BIM, LLMs present a novel opportunity to simplify access to complex model data. By integrating LLMs with backend systems that store BIM data—particularly graph-based databases such as Neo4j—it becomes possible to create intelligent assistants that allow users to query and interact with building

models using plain language. This paradigm shift could significantly improve user experience, broaden adoption, and unlock new workflows for design validation, spatial analysis, and semantic search.

This thesis investigates the integration of a Large Language Model with a Neo4j graph database to develop an AI-powered BIM assistant. The goal is to enable users to perform advanced semantic searches and rule checks on BIM data using natural language inputs. To contextualize this work, the following chapters examine the structure and semantics of BIM data, the limitations of traditional data storage models, and the architectural design of the proposed assistant system. Understanding the challenges of querying BIM semantics is essential to appreciating the motivation and technical approach of this thesis.

1.2 BUILDING INFORMATION MODELS

The AEC industry is undergoing a significant transformation driven by digital technologies that enhance collaboration, efficiency, and data management throughout the building lifecycle. At the forefront of this transformation is Building Information Modeling (BIM), a process that involves the generation and management of digital representations of physical and functional characteristics of built assets. BIM serves as a shared knowledge resource, providing a reliable basis for decision-making from the earliest conception through to demolition [8].

Unlike traditional two-dimensional drawings, BIM encompasses multi dimensional data, integrating geometry (3D), time (4D), cost (5D), and other critical information such as sustainability and facility management. This comprehensive approach facilitates improved visualization, coordination, and analysis, leading to enhanced project outcomes.

Central to the effective implementation of BIM is the need for interoperability among diverse software applications used by various stakeholders. The Industry Foundation Classes (IFC) address this need by providing an open, platform-neutral data schema developed by buildingSMART. IFC, regardless of its challenges, enables the seamless exchange and sharing of BIM data across different software platforms, ensuring consistency and collaboration throughout the project lifecycle.

The adoption of IFC has been instrumental in promoting openBIM practices, fostering greater collaboration, and reducing data silos within the AEC industry. For instance, in Finland, the use of IFC is mandated for publicly aided building projects, reflecting a commitment to standardized, interoperable data exchange.

1.3 BIM SEMANTICS

As mentioned above, building models contains both geometrical and semantic information. Geometry defines the physical shape and spatial characteristics of building components, whereas semantics provide meaningful descriptions that help identify and classify these components within the context of a building

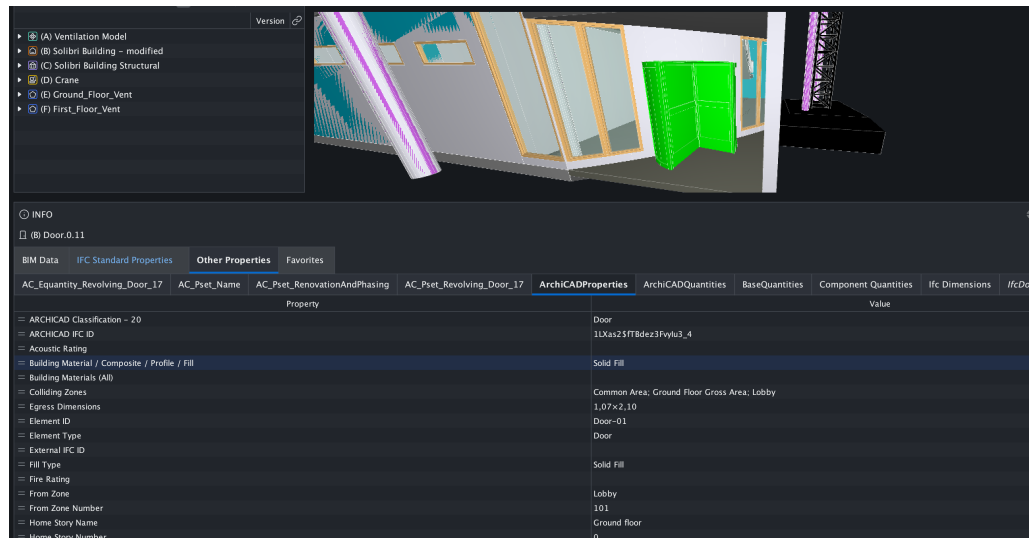


Figure 1.1: Screenshot from Solibri showing a revolving door and its semantics

model. In simple terms, semantics describe what an element is and what its role is in the building.

Consider, for example, a door in a building model. Beyond its geometry, which includes width, height, and depth, the door can carry a wealth of additional semantic information that characterizes it more precisely. These may include attributes such as fire rating, material type, whether it is an external door, the type of hinge used, and its precise location in 3D space. Additionally, the door may have defined relationships with other elements in the model, such as being part of a specific wall or zone, or connected to a specific access path or emergency route. These semantic attributes are critical for various analyses and simulations, such as energy modeling, safety evaluations, and code compliance checks.

Tools like Solibri allow users to access this information easily through their interface, where details about any element can be visualized on demand (as illustrated in 1.1). However, as the complexity of the building model increases, with hundreds or thousands of interconnected elements, navigating and extracting relevant information can become challenging. Filtering components based on specific semantic properties or conducting rule-based checks often requires a deep understanding of both the building domain and the software tool itself. Consequently, leveraging the full capabilities of Solibri typically necessitates the involvement of a BIM expert (see 1.2).

Solibri supports a number of commonly used BIM file formats, but the most widely used and supported format across industry authoring tools is the Industry Foundation Classes (IFC). IFC is a standardized data model designed to describe the built environment in a platform-neutral way. It encapsulates both geometric and semantic information, enabling consistent data exchange and interoperability between different BIM tools and stakeholders. This, in turn, supports the automation of design and compliance workflows, making it a foundational element in the digital transformation of the construction industry.

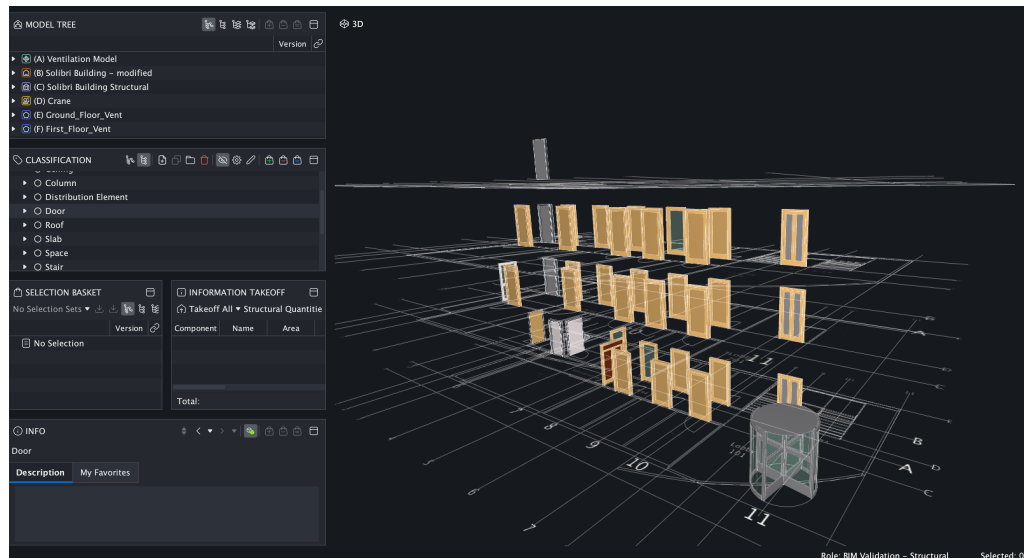


Figure 1.2: Isolating all doors in a model

1.4 OBJECTIVE

The primary objective of this research is to design and implement an AI-driven assistant that enables users to interact with Building Information Modeling (BIM) data using natural language. Specifically, the system aims to translate user questions or commands into structured queries that can retrieve relevant information from a database and execute predefined quality assurance checks, without requiring the user to explicitly define technical criteria or navigate complex user interfaces.

This approach is motivated by the observation that interacting with BIM data typically involves detailed configuration, domain-specific knowledge, and familiarity with specialized filtering mechanisms. For example, in Solibri, users must configure rule parameters, property filters, and selection views to perform quality checks or inspect specific model elements. These steps, while powerful, can act as barriers for users who are not deeply familiar with the software or the underlying data structures. By introducing a natural language interface, the system seeks to lower the threshold for accessing and understanding BIM data, making advanced querying and checking workflows more accessible and efficient.

The assistant developed in this thesis leverages Large Language Models (LLMs) to interpret user prompts and convert them into database queries that are both syntactically correct and semantically meaningful. In addition to facilitating flexible searches, the system supports basic quality assurance checks by extracting key criteria from user input and mapping them to appropriate rule logic within the database. This enables non-expert users to perform checks such as "Are all doors accessible for wheelchairs?" or "List all load-bearing walls on the second floor," using natural language rather than manual configuration.

This research builds upon Solibri's existing capabilities, particularly its mature Industry Foundation Classes (IFC) interpreter. Solibri has been developed and refined over more than 25 years and offers comprehensive support for reading, inter-

preting, and visualizing IFC files. It constructs 3D representations of buildings, derives geometric relationships, and exposes semantic data such as element types, classifications, and property sets. These functionalities form the foundation of the system explored in this thesis.

To enable AI-assisted querying, the thesis presents a workflow for extracting and transforming Solibri's internal BIM data into a format suitable for graph-based querying. This involves preprocessing and cleaning the data, converting it into a graph structure, and storing it in a Neo4j graph database. The graph model preserves both geometric and semantic relationships, making it suitable for flexible and efficient traversal. On top of this database, an AI agent is implemented to interpret user input, generate Cypher queries, and return results in formats interpretable by both machines and humans. This assistant system is designed to complement and extend Solibri's existing toolset with a novel, user-friendly interface for interacting with complex BIM data.

1.5 RESEARCH SCOPE AND METHODOLOGY

The scope of this research is to build a proof of concept with limited capabilities, such as retrieving building elements based on their attributes, and relations to other elements in the same model, executing simple checks such as clash detection, distance, and existence checks.

While building a full user interface and defining all user interactions is out of scope, the system will expose an Application Programming Interface (API) to facilitate structured and scalable input and output. This API will allow users to send natural language queries and receive results in a machine-readable format, ensuring seamless integration with other tools or future interfaces.

The development of a robust and reliable AI agent capable of retrieving information from a database based on natural language user prompts requires a well-structured and systematic methodology. This research follows a multi-step process to ensure accuracy, efficiency, and relevance in the AI agent's responses.

The first step involves data modeling and database selection, which encompasses evaluating different database technologies and structuring the Building Information Modeling (BIM) data in a way that supports efficient and meaningful queries. The chosen data model must align with the semantics of BIM data and support scalable, flexible retrieval.

Next, a data cleaning and importing pipeline is established to prepare the BIM data for use. This step focuses on validating, normalizing, and importing the data into the selected database while preserving its integrity and eliminating inconsistencies that could compromise the AI agent's performance.

Following this, the key objectives and functional boundaries of the AI agent are clearly defined. This includes specifying the types of queries the agent should be able to handle, the level of detail expected in the responses, and constraints to prevent misuse or overreach of the system's capabilities.

Prompt engineering plays a critical role in translating user intent into accurate

and context-aware database queries. Carefully designed prompts help guide the language model to produce responses that are not only relevant and accurate but also safe and aligned with the expectations of the domain.

While techniques such as Retrieval-Augmented Generation (RAG) offer promising avenues for enhancing AI-generated responses by incorporating real-time document retrieval, they fall outside the scope of this thesis and are identified as an area for future exploration.

The subsequent sections of this thesis will examine each of the aforementioned steps in detail, outlining both the theoretical foundation and practical implementation involved in building the AI-driven BIM assistant.

1.6 SIGNIFICANCE OF THIS RESEARCH

This research investigates the intersection of Artificial Intelligence (AI) and Building Information Modeling (BIM), with a particular focus on the application of Large Language Models (LLMs) to enhance traditional workflows in BIM environments. By enabling natural language interaction, LLMs provide a novel interface layer between users and complex BIM data structures, allowing professionals to query, inspect, and validate models without the need for extensive technical expertise or manual configuration.

The integration of LLMs into the BIM workflow introduces significant advantages. It simplifies user interaction, reduces cognitive load, and lowers the entry barrier for non-expert users. Routine tasks, such as filtering elements, checking compliance with basic design rules, or extracting specific model information, can be executed more intuitively and with fewer errors. This intelligent assistance improves both efficiency and reliability, offering faster feedback loops during the design and review stages of construction projects.

In addition, the AI-powered assistant developed in this thesis contributes to a broader vision of accessible and intelligent design tools. Using semantic understanding, graph-based data representation, and conversational AI, the system represents a step toward more user-friendly and adaptive construction software. Ultimately, the research demonstrates how LLMs can serve as a bridge between technical BIM platforms and human users, paving the way for smarter, more inclusive digital tools in the AEC industry.

2 KEY TECHNOLOGIES

2.1 GRAPH AND GRAPH DATABASE

At a fundamental level, a graph is a mathematical structure used to model pair-wise relations between objects. A graph consists of a set of vertices (or nodes) that represent the objects, and a set of edges that represent the relationships between them [18]. This simple but powerful abstraction is ideal for representing complex, networked systems where the connections and relationships between entities are as important as the entities themselves. In such a model, both vertices and edges can be assigned attributes or properties, allowing for a rich, multi-layered description of the system being modeled. For example, in a building model, a vertex could represent a room with properties like 'name' and 'area', while an edge could define the relationship of the room with other entities in the building, such as a door or an adjacent room.

The true power of representing data as a graph lies in the ability to analyze the structure of these connections. Using graph theory, it becomes possible to perform complex queries and analyses that are impractical with other data structures. This includes pathfinding algorithms to find the shortest or optimal route between two nodes (e.g., for an emergency exit path or a utility line), centrality analysis to identify the most critical nodes in a network (e.g., a key structural component or a central networking hub), and community detection to find clusters of densely connected nodes (e.g., identifying functional zones within a building). The application of these graph-based analytical methods to Building Information Models has been shown to be a potent way to explore and understand the intricate topological relationships inherent in building design and construction [28].

Graph databases organize information as nodes (entities) connected by edges (relationships), representing data as a network rather than tables. This model naturally captures highly interconnected structures like Building Information Models (BIM). In recent years, graph databases have gained popularity for managing complex data because they allow "efficient handling and analysis of interconnected data". In a property-graph model (used by Neo4j and similar systems), nodes can carry labels and properties, and edges carry types and attributes. This makes it easy to encode semantic relationships (e.g., "Wall is part of Storey" or "Room A is adjacent to Room B") directly in the data model. Thus, graph-based storage provides an intuitive representation of BIM data, in which entities (walls, doors, spaces, etc.) are nodes and spatial or hierarchical links between them are first-class edges. [27] note that "graph databases have emerged as a powerful solution for managing highly connected data, providing substantial benefits in terms of flexibility, query performance, and data exploration capabilities. 2.1 illustrates how a simple graph, nodes, and edges, are represented by Neo4j.

Graph databases offer several key advantages over traditional relational databases when dealing with hierarchies and connectivity:

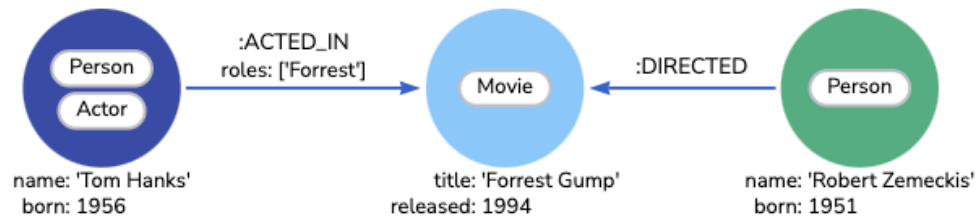


Figure 2.1: Graph representation with Neo4j syntax

- In a graph DB, navigating from one node to another via an edge is a simple pointer follow. [26] explains that graph traversal is based on these direct links, so “following a pointer is a constant time operation, whereas joining tables is more complicated and slower”. In contrast, a relational query that retrieves connected entities often requires multiple expensive JOIN operations (e.g., joining tables for rooms, floors, buildings, etc.). Graph queries avoid this overhead by walking the stored edges directly.
- Relational schemas can struggle with recursive queries over a hierarchy (for example, finding all sub-spaces under a building). Graph databases excel at such tasks: queries simply traverse the graph and can naturally follow arbitrary-length paths. Previous benchmarks show Neo4j outperforming SQL databases on simple connected queries. [27] likewise note that relational systems struggle with complex queries that involve multiple joins and aggregations, whereas graph databases excel at traversing relationships. This means connectivity-based queries (like finding all components within a spatial hierarchy) run more directly and often faster in a graph.
- Graph models allow the schema to evolve dynamically. Nodes and relationships can be added or retyped on the fly without rebuilding entire tables or schemas. In contrast, a relational database requires a predefined table structure, and altering it can be costly. As [27] points out, graph DBs support dynamic schema evolution, enabling new entity types or relations to be introduced without affecting the existing data. This agility is valuable in domains like BIM, where the data model may grow or change (new element types, new relationship categories, etc.) over time.
- Many analytical queries on networks (e.g., shortest paths, connectivity, pattern matching) map directly onto graph operations. Graph databases often include built-in algorithms for traversals, centrality, clustering, etc., which can be applied directly on the stored data. Relational databases typically require complex SQL or external tools to perform similar analyses.

In summary, for hierarchically structured and richly connected datasets like BIM, graph databases eliminate the need for complex join logic and yield faster connectivity queries. They are designed to scale to large, relationship-dense graphs, e.g., Neo4j can handle billions of nodes and relationships while maintaining ACID transactions, thus providing a practical platform where relational models become unwieldy.

Building Information Models inherently contain hierarchies (e.g., campus -> site -> building -> floor -> room) and spatial relationships (spaces adjacent or contained).

Graph databases model these naturally: each spatial or containment relation becomes an edge in the graph. For example, spatial hierarchy (a building part-of relation) is encoded as a chain of connected nodes, and space boundaries (which element bounds which space) are direct edges. [41] explicitly enumerates key IFC relationships – “spatial structure” (hierarchy of spaces), “spatial containment” (elements assigned to spaces), and “space boundary” (elements bounding spaces) – and shows that these map cleanly onto graph edges. In a graph database, retrieving all elements in a space simply follows the containment edges; finding adjacency or accessibility is a path query in the spatial graph.

This approach has been demonstrated in practice on BIM data. [40] built an IFC-Graph representation of a BIM (Industry Foundation Classes) model and developed a Cypher-based query tool (GQL4BIM). They found that complex building relationships could be queried directly in the graph: for instance, spatial structure queries (e.g., “which spaces lie under this floor?”) and space boundary queries (“which elements bound this room?”) were answered by simple Cypher patterns. The results show that graph queries can navigate multi-level hierarchies and adjacency relationships more straightforwardly than equivalent relational queries. Similarly, [41] compared BIM querying via Neo4j/Cypher to a standard IFC parsing library. They observed that Cypher queries returned the same entities but with stable, efficient timings: for example, retrieving all wall objects in a model took under 2ms on average, whereas the legacy method slowed down as the result set grew.

Graphs also excel in performance for connectivity-based queries. In the [41] study, graph queries traversing spatial hierarchies completed in well under a few seconds, even for moderately sized structures: querying a 97-node spatial structure path took on average 1.85s, and larger containment queries (458 nodes) took 0.48s. These results indicate that Neo4j handled multi-hop, connectivity-intensive queries gracefully. By contrast, a relational approach would require multiple joins and recursion in SQL for the same tasks, generally incurring higher overhead.

Additionally, a comprehensive benchmark by Kotiranta et al. compared Neo4j, MySQL, and MariaDB using both simple and complex queries, with and without indexing and optimization. They confirmed graph databases outperform relational systems in simple traversals, while modern relational databases (especially MariaDB) excel in more complex or aggregation-heavy use cases—underscoring the crucial role of indexing and optimization for each paradigm [25].

Neo4j is a leading open-source graph database that exemplifies these advantages. It stores native graph data with ACID transactions, meaning updates (adding new BIM elements or relationships) are safe and consistent. Neo4j’s Cypher query language is highly expressive for graph patterns (e.g., specifying multiedge paths) and is optimized under the hood for graph traversal. In practice, this means a query like

```
MATCH (floor:IfcSpatialStructureElement) -[:IfcRelAggregates*]
->(room:IfcSpace) RETURN room
```

can find all rooms under a given floor in one statement, leveraging indexed relationships.

Empirical studies reinforce Neo4j’s effectiveness. [26] summarizes prior comparisons and notes that “graph databases, especially Neo4j, were compared with the traditional SQL databases, indicating that graph databases possess better performance than relational databases” on typical connected-data queries. Neo4j’s native pointer-based traversal ensures that, for many BIM queries, execution is dominated by simple graph walks rather than expensive table lookups. For example, the [41] experiments found that Neo4j/Cypher queries responded much faster on large result sets compared to an IFC file parser written in C++.

Overall, Neo4j (and similar graph platforms) provide a compelling backbone for BIM storage and query. Hierarchical spatial data (buildings, floors, rooms) map directly to node-edge structures, and complex relationships (containment, adjacency, inheritance) become native graph links. As [40] concludes, this enables effective building of information queries, including digital twin and knowledge-graph applications, that would be cumbersome with traditional SQL schemas. In sum, graph databases transform the handling of BIM’s highly interconnected data: relationships are no longer derived by joining tables but are inherent in the model, yielding simpler queries and often much better performance for connectivity-heavy use cases [27] [26] [40]

2.2 SERVERLESS COMPUTING

Serverless computing, often referred to as Function-as-a-Service (FaaS) [23], is a modern cloud-native architectural paradigm that enables developers to write and deploy small, stateless functions that are triggered by specific events. In this model, the cloud provider dynamically manages the allocation and scaling of resources, eliminating the need for manual server provisioning or infrastructure management. Each function is designed to execute a discrete unit of logic, often triggered by HTTP requests, database updates, file uploads, or messaging events, and terminates as soon as its task is completed. This makes FaaS particularly suitable for lightweight operations that benefit from rapid, event-driven execution and fine-grained billing based solely on compute time and resource consumption. As [32] explains, the developer’s primary responsibility is limited to defining the core logic and APIs, while the underlying platform abstracts away operational concerns such as scaling, fault tolerance, and resource cleanup.

The serverless model aligns well with applications that require high elasticity and modularity, such as data pipelines, microservices, or AI-driven backend services. In the context of this thesis, serverless functions are employed to ingest, clean, and import BIM data into a graph database, leveraging cloud-based services like AWS Lambda and S3. AWS Lambda functions are commonly used for serverless tasks [7], while S3 provides reliable object storage [6]. This architecture ensures that the system can efficiently process incoming IFC-derived data files on demand, scaling up when new models are uploaded and releasing resources when processing is complete. The lightweight and stateless nature of FaaS components also complements the use of a graph database like Neo4j, which remains persistently available and optimized for complex relational queries. When paired together, serverless computing and graph databases offer a highly responsive and

scalable backend suitable for AI-enhanced systems, where user prompts must be interpreted and executed with low latency and minimal overhead.

2.3 LARGE LANGUAGE MODELS (LLM)

Large Language Models (LLMs) are deep neural language models that typically use the Transformer architecture [2]. The Transformer’s multi-head self-attention mechanism allows the model to weigh the importance of all tokens in an input sequence simultaneously, enabling highly parallel computation and the effective capture of long-range context. As a result, modern LLMs (e.g., BERT, GPT, T5) consist of stacks of Transformer layers with billions, or even trillions, of parameters, pre-trained on massive text corpora sourced from the public web, books, and other data, often measuring in the terabytes. These models are first trained in a self-supervised manner and then adapted to specific tasks via fine-tuning.

Training an LLM generally proceeds in two stages. In the first stage, the model is pretrained on very large unlabeled text datasets using objectives like autoregressive next-token prediction or masked-token prediction. This self-supervised pre-training builds broad linguistic and world knowledge into the model. In the second stage, the pre-trained model is adapted using labeled data. A key technique in this stage is Reinforcement Learning from Human Feedback (RLHF), where human preferences between different model outputs are used to train a reward model. This reward model, in turn, fine-tunes the LLM to produce outputs that are more helpful, harmless, and aligned with user intent [30]. This process is often part of a broader strategy of instruction-tuning, which aligns the model with human-style instructions. These adaptation steps can substantially improve performance, to the point where a smaller, instruction-tuned model can be qualitatively preferred over a much larger, base model that lacks such alignment [30].

The resulting LLMs exhibit powerful general-purpose capabilities. For instance, [5] showed that GPT-3 (175 billion parameters) achieves strong few-shot performance on diverse tasks (translation, question answering, cloze completion, arithmetic puzzles, etc.) without any additional training. Many advanced abilities, such as multi-step reasoning or complex pattern recognition, appear to emerge only in very large models, a phenomenon termed emergent abilities [3]. In addition to text tasks, LLMs can generate code: for example, OpenAI’s Codex model (a GPT family model fine-tuned on GitHub code) solved 28.8% of problems in a standard program synthesis benchmark, a task where the base GPT-3 solved 0% [17].

The trajectory of scaling and architectural refinement has culminated in models like GPT-4 [29]. While its exact architecture and parameter count are not public, it is understood to be substantially larger and more capable than its predecessors. A significant leap for GPT-4 is its native multi-modality, allowing it to process not just text but also image inputs, integrating visual information into its reasoning process. This has led to state-of-the-art performance across a wide array of professional and academic benchmarks, from passing the Uniform Bar Exam with a score in the top 10% of test takers to demonstrating sophisticated reasoning and instruction-following capabilities far beyond those of previous models [29].

These LLM capabilities have direct relevance for graph databases like Neo4j. Instead of writing Cypher queries by hand, a user can pose a question in natural language and let an LLM generate the corresponding Cypher statement. This synergy has created a rapidly growing subfield of research. Recent work by [22] demonstrates that transformer LLMs can translate English queries into Cypher end-to-end. For example, [16] used a Transformer model to convert NL questions into CQL patterns (without intermediate templates), and [4] prompts ChatGPT (providing the graph schema) to output correct CQL statements. Out-of-the-box LLMs may make mistakes on nuanced queries, but fine-tuning on domain-specific NL–Cypher pairs significantly improves accuracy. In effect, an LLM-powered interface turns natural-language questions into executable graph queries, lowering the barrier to Neo4j data access: users simply ask a question in English and receive a ready-to-run Cypher query in return.

2.4 INDUSTRY FOUNDATION CLASSES (IFC)

The IFC standard defines a structured, machine-readable schema for sharing data of the built asset industry [11]. An IFC often includes geometry, semantics, and relationships of a building element. IFC acts as an interoperability standard between different BIM authoring tools, enabling seamless data exchange across the AEC industry.

The Resource Layer encompasses fundamental definitions such as geometry, materials, and units, serving as the foundational building blocks for higher-level abstractions. The Core Layer introduces general entity definitions with globally unique identifiers, establishing the basic structure and relationships within the model. The Interoperability Layer defines entities applicable across multiple disciplines, promoting inter-domain data exchange. Finally, the Domain Layer contains discipline-specific entities, facilitating intra-domain information exchange and specialized modeling needs [37].

Solibri is a leading tool for interpreting IFC files, offering features such as rule-based validation, clash detection, and compliance checks. It uses parameterized rules to perform quality assurance, allowing users to verify models against criteria like building codes and design guidelines. However, effectively filtering and querying IFC data within Solibri often demands deep expertise, posing challenges for users without specialized training.

To enhance accessibility and flexibility in querying IFC-based BIM data, this research explores importing IFC-derived data into a graph database, such as Neo4j. By representing building elements as nodes and their relationships as edges, graph databases naturally model the interconnected nature of BIM data. This structure allows for more intuitive and efficient querying, especially when combined with AI-driven natural language interfaces. Users can pose queries in everyday language, which are then translated into structured queries (e.g., Cypher) to retrieve relevant information from the graph database. This approach aims to democratize access to complex BIM data, reducing the reliance on specialized knowledge and making data retrieval more user-friendly.

2.5 JSON AND GLTF FORMATS

JSON (JavaScript Object Notation) is a lightweight, text-based data interchange format widely adopted for its simplicity and human-readability. Its structure, based on key-value pairs, facilitates seamless data exchange between systems, making it particularly suitable for web-based applications. In the context of Building Information Modeling (BIM), JSON has been utilized to represent complex data structures, enabling efficient serialization and transmission of information. [1] introduced the ifcJSON schema, a JSON serialization of the Industry Foundation Classes (IFC) data model, to support web-based BIM data exchange and improve interoperability among BIM tools.

In this thesis, JSON is employed to transmit BIM semantic data compiled in the Solibri desktop software. This approach focuses on conveying essential information such as object types, names, relations to other entities, quantity calculations, and other interpretable attributes while excluding detailed geometrical data. The decision to omit geometrical information is based on considerations of data storage efficiency and the scope of the AI agent's functionality, as incorporating full geometries would significantly increase the workload and storage requirements with minimal impact on the results. By utilizing JSON for semantic data exchange, the system benefits from a lightweight and efficient format that aligns with the needs of AI-driven BIM data processing.

GLTF (Graphics Library Transmission Format) is an efficient, open-standard format for 3D model representation [24]. It is also optimized for web-based and real-time rendering, making it a preferred format for visualizing BIM elements. Although geometries are important for visualization purposes, utilizing them for the purpose of building an AI agent was decided out of scope for this thesis work, as it would significantly increase the workload and data storage price for insignificant improvements in the results. In the mentioned JSON files, however, information such as object location, bounding box coordinates, and other interpretable data points is included to enrich the semantics even further. 2.2 shows a sample building model extracted from the Solibri desktop.

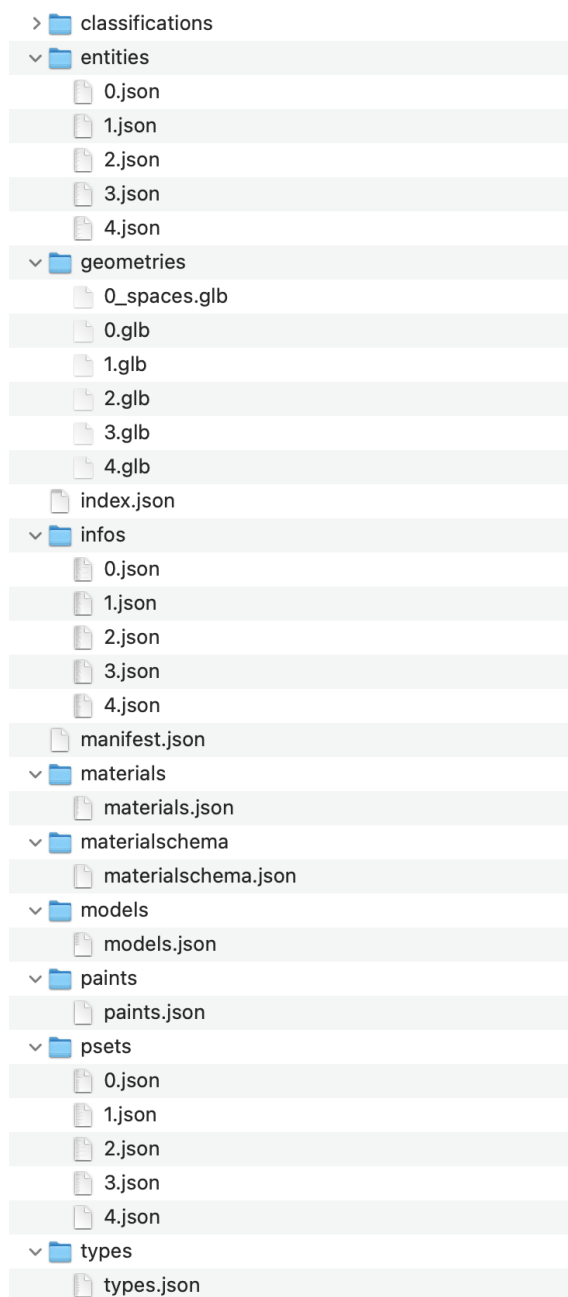


Figure 2.2: A sample from an extracted zip file demonstrating the file structure

3 PAST ACHEIVEMENTS

3.1 SOLIBRI DESKTOP

Existing tools for exploring BIM semantics, such as Solibri, have played a crucial role in ensuring model compliance, clash detection, and quality assurance. Over the years, Solibri has evolved into a comprehensive platform that allows BIM experts to inspect and validate building models through a combination of rule-based filters and configurable checks. These capabilities ensure that models meet industry standards before construction, reducing errors and costly rework. While this rule-based approach is powerful, it remains highly dependent on user expertise. Configuring rules and filters requires a deep understanding of both the software and BIM standards, making it difficult for new or non-expert users to fully leverage Solibri's capabilities. As the complexity of building models grows, so does the challenge of setting up effective queries and checks.

Current solutions for BIM validation primarily rely on predefined rule sets and manual configurations, which often lack adaptability to natural language inputs. Users must navigate complex rule hierarchies, understand property definitions, and manually apply filtering logic. This traditional method, although robust, creates a barrier for those unfamiliar with Solibri's interface and syntax.

The primary objective of this research is to ultimately bridge the gap between Solibri's rule-based approach and a more user-friendly AI-powered workflow. This work explores how an LLM-enhanced system can act as an intermediary, enabling users to perform queries for data retrieval in various forms and execute simple quality assurance checks by writing a prompt describing their needs.

3.2 DATA EXPORTER

Solibri uses a proprietary file format (SMC) to store interpreted and merged BIM files, optimizing performance, caching, and sharing within the software. However, SMC files are not natively recognized or usable by third-party applications. Accessing the information within an SMC file is crucial for performing operations on BIM data without the need to reinterpret the original source files, improving efficiency and workflow integration.

A previous project work at Solibri explored a method for exporting SMC files into JSON and GLTF formats, enabling external applications to access and utilize Solibri-processed BIM data. This approach provided a structured way to extract both semantic and geometric information, facilitating further analysis without requiring Solibri's proprietary environment. Building on the mentioned work, this research aims to leverage graph databases and AI-driven querying to enhance how users interact with BIM data as simply and efficiently as possible.

3.3 PREVIOUS AI APPROACHES IN BIM

Utilizing AI in BIM, although new, is not uncommon to see. With the exponential growth of AI and large language models over the past few years it has become an inevitable objective for every industry to adopt AI one way or another to simplify their processes, make their efforts more efficient, and reduce human error. To mention a few of the past research to have used large language models to expand the capabilities of a BIM authoring tool, the following works are the most related to this research.

BIM-GPT is a prompt-based virtual assistant framework that combines BIM with GPT technologies to facilitate natural language-based information retrieval. The framework employs a prompt manager and dynamic templates to generate prompts for GPT models, enabling the interpretation of natural language queries and summarization of retrieved information. Their approach achieved high accuracy rates in classifying natural language queries, demonstrating the effectiveness of integrating LLMs for BIM information retrieval [39].

An LLM-based multi-agent framework was designed to generate 3D building models from natural language instructions. This system coordinates multiple LLM agents to transform textual user input into imperative code that interacts with BIM authoring tools, producing editable BIM models with detailed layouts and semantic information. The framework was integrated into the BIM authoring software Vectorworks, showcasing the potential of “modeling by chatting” to streamline the design process [19].

In another study, a copilot framework within BIM authoring tools was developed, utilizing an LLM-based autonomous agent. This agent assists designers by understanding design intentions from natural language inputs and autonomously executing modeling tasks. Implemented as a prototype in Vectorworks, the system demonstrated significant potential in design automation and intelligent human-machine interaction within BIM environments [20].

Collectively, these studies underscore the transformative potential of AI and LLMs in BIM, offering innovative solutions for information retrieval, model generation, real-time interaction, and intelligent assistance within the construction and design industries.

While previous studies have explored the integration of Large Language Models (LLMs) into Building Information Modeling (BIM) for tasks such as information retrieval and real-time model interaction, the current research distinguishes itself by focusing on the development of an AI agent that translates natural language prompts into Cypher queries executed on a Neo4j graph database. This study eliminates the need for training or fine-tuning a model before using it, reducing the training overhead and execution time.

This work heavily focuses on laying a strong foundation for the AI agent to rely on, furthering the accuracy and repeatability of the AI agent. This includes comprehensive data conversion and cleaning processes to maintain data integrity, the design of an efficient database architecture tailored for BIM data within Neo4j, and the creation of a streamlined data insertion pipeline. Collectively, these founda-

tional elements support the AI agent's ability to generate precise Cypher queries, thereby enhancing the efficiency and usability of BIM data retrieval and analysis.

4 METHODOLOGY AND TECHNICAL APPROACH

Developing a system intended for deployment in a production environment necessitates a comprehensive and methodical approach. This process encompasses several critical phases: designing a robust system architecture, meticulously evaluating and selecting appropriate tools and third-party software based on defined criteria, and executing thorough implementation and testing procedures. The foundation of a reliable system lies in its architecture. A well-conceived system architecture ensures scalability, maintainability, and resilience. Key considerations include modularity, which allows for components to be developed and updated independently; scalability, to handle increasing loads; and fault tolerance, to maintain functionality in the event of component failures. Adhering to established architectural frameworks and best practices can guide the development of systems that meet these criteria. The selection of tools and third-party software is equally pivotal. An effective evaluation process involves defining clear requirements, assessing compatibility with existing systems, and analyzing factors such as performance, security, support, and cost. Engaging in pilot testing or proof-of-concept implementations can provide valuable insights into how well a tool or software aligns with the system's objectives and constraints. Implementation and testing are the final, yet crucial, stages. A structured implementation plan ensures that the system components are seamlessly integrated. Comprehensive testing, including unit, integration, and system testing, is essential to validate that the system performs as intended under various conditions. Incorporating automated testing and continuous integration practices can improve the reliability and efficiency of the development process. Each of these phases significantly influences the overall effectiveness and reliability of the system. Neglecting any aspect can lead to suboptimal performance or system failures. The subsequent sections will delve into each phase in detail, outlining the methodologies and considerations essential for building a robust and efficient assistant system.

4.1 SYSTEM ARCHITECTURE DESIGN

The system is designed to continuously process incoming BIM files and import them into the database, enabling end users to utilize the system as soon as their data is processed. The demonstrated system architecture diagrams in 4.1 show a series of key components and services necessary to maintain scalability, reliability, and efficiency.

At a high level, the system architecture is composed of several key modules, each serving a specific role in managing, processing, and querying BIM data. These components work together to support a seamless and secure flow of information, from raw IFC file uploads to AI-assisted data retrieval. The entire system is designed with modularity in mind, using cloud-native infrastructure to ensure scalability, efficiency, and robust access control.

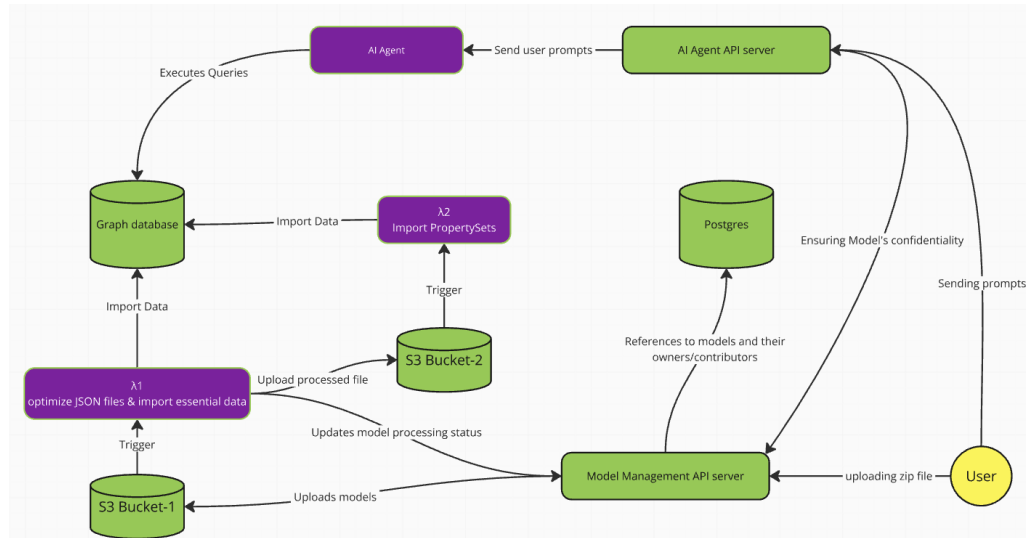


Figure 4.1: System Architecture Design

The Model Management API Server functions as the central control layer, handling administrative tasks such as uploading model data, managing user permissions, and enforcing data confidentiality through authentication and authorization mechanisms. This service is critical to the integrity and security of the system, as it ensures that users can reliably access their own data while protecting against unauthorized access. It acts as the user-facing gateway to the BIM model life-cycle, supporting core operations needed to store and retrieve building models securely.

Incoming BIM data is stored in Amazon S3 buckets, which serve as the primary storage layer. BIM models are uploaded as compressed ZIP archives and organized using unique identifiers that are synchronized with records in the Postgres database. This prevents mixups between projects and ensures consistency across storage and query operations. Once files are uploaded, preconfigured S3 event triggers activate specific AWS Lambda functions. A secondary S3 bucket holds auxiliary metadata and property sets, which are either used for enriching the database or accessed by other services such as front-end applications (though front-end processing is beyond the scope of this thesis).

The system uses two Lambda functions to manage the transformation pipeline. The first function ($\lambda 1$) processes the uploaded ZIP file by extracting, aggregating, and cleaning multiple JSON model files. These often include hundreds of models, each comprising at least three separate files. $\lambda 1$ also uploads the geometry-related assets to a secondary S3 bucket for direct rendering or visualization. The second function ($\lambda 2$) performs insertion of additional metadata into the graph database, completing the process in a distributed fashion. This two-step approach is designed to bypass the AWS Lambda execution time limit of 15 minutes, ensuring that even large datasets can be handled reliably.

The processed BIM data is inserted into the graph database, which, due to its inherent structure of nodes and relationships, is well suited to represent interconnected building elements. This database serves as the backend for complex, connection-oriented queries such as spatial relationships, component aggrega-

tions, and semantic metadata retrieval.

To make querying this data intuitive and accessible, the system incorporates an AI Agent powered by a large language model (LLM). This component interprets natural language user prompts and translates them into structured Cypher queries, which are executed against the graph database. The AI agent enables users, including non-experts, to retrieve building information without needing technical knowledge of the database schema or query language. Communication with the AI agent is managed through the AI-Agent API Server, which acts as a structured and secure gateway for integrating the language model with other services. It handles authorization, input validation, and response formatting to ensure reliability and maintainability.

In general, the system architecture uses native services from AWS - including Lambda and S3 alongside AI tools based on Neo4j and LLM to deliver a scalable, secure, and user-friendly platform for BIM data processing and retrieval. By integrating traditional data engineering components with advanced AI, the platform offers a robust solution for flexible, natural-language-driven exploration of complex building models.

4.2 SELECTING A SUITABLE DATABASE

Efficiently storing and querying BIM data requires selecting a database that can handle semi-structured metadata and complex relationships between building elements. Various database types were considered, each offering different advantages and trade-offs. As mentioned before, BIM data is made up of highly nested structures with complex relationships to other elements in the model (4.2). Therefore, an efficient database with a proper data model is a necessity in this system that cannot be compromised.

Database selection for such systems requires extensive research with clear criteria. An optimal database to handle complex and non-deterministic data schema was decided to satisfy the following criteria:

- flexibility and scalability: As the load varies, it should be possible to scale up and down accordingly to optimize cost over utilization
- Complex Query performance: This application requires a database that is efficient in data retrieval since the data records are imported once and never get updated. Therefore, retrieval capability is highly demanded over write efficiency.
- Cost: Beyond technical capabilities, cost and resource efficiency played a significant role in selecting the right database, however, for the sake of research, the cost of the database was not prioritized over other criteria.
- AI Support: Given the system's reliance on AI-driven querying and analysis, additional considerations included similarity search capabilities and potential built-in AI features that could enhance query performance.

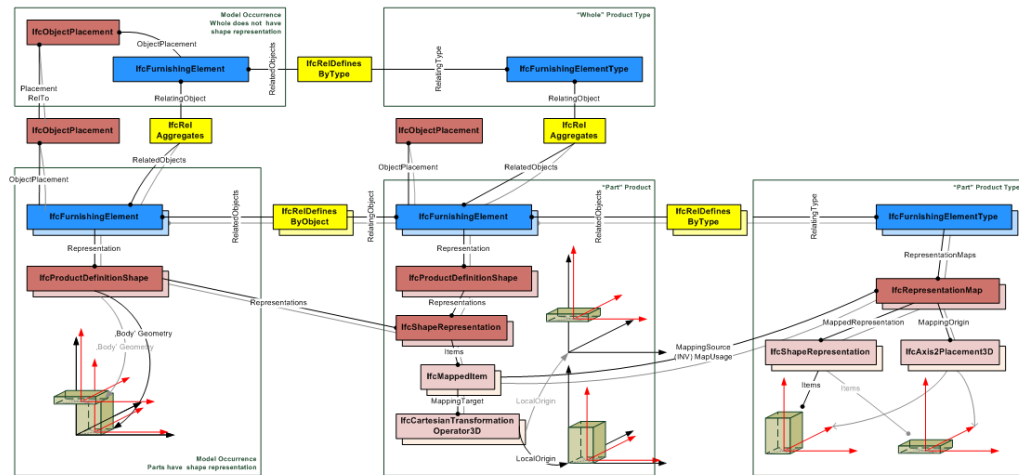


Figure 4.2: Illustrating relationships in an IFC file provided in buildingSmart.org

4.2.1 RELATIONAL DATABASES

Relational databases store data in tabular structures with predefined schemas, where each table consists of rows and columns representing entities and their attributes. This model is well-suited for highly structured and transactional data. It provides consistency, support for ACID-compliant operations, and compatibility with a wide range of existing tools and optimizations [21].

However, applying the relational model to Building Information Modeling (BIM) data reveals fundamental limitations. BIM data is inherently hierarchical, nested, and interconnected. Building elements exist within complex spatial and semantic structures. For example, a wall may be part of a room, belong to a specific floor, be related to other walls in a group, and carry multiple property sets that vary based on its classification and use.

To represent this in a relational schema, each concept must be decomposed into normalized tables, and relationships between entities are handled using foreign key constraints. Querying such structures typically involves multiple JOIN operations across several tables, which introduces significant overhead, especially as model complexity grows. A typical model may contain over 50,000 individual elements. When their relationships and property sets are taken into account, the number of stored records may grow by more than 20 times, resulting in over 1,000,000 records per model, depending on the data model. Moreover, a single project archive can contain between 1 and 100 such models.

Although modern relational databases such as PostgreSQL and MySQL can manage millions of records, the depth and frequency of JOIN operations required for BIM queries lead to performance bottlenecks. This is particularly problematic in tasks such as rule checking, clash detection, or semantic filtering, where traversing deep and complex relationships is necessary [33].

Due to these issues, relational databases are generally not the optimal choice for managing BIM data at scale. Alternatives such as graph databases or document-based systems offer more natural representations of the data and can provide

better performance and flexibility [9], [31].

4.2.2 DOCUMENT DATABASES

Document databases, such as MongoDB, store data in flexible, JSON-like (or BSON for MongoDB) documents, allowing for the storage of hierarchical and semi-structured data. This flexibility is a significant departure from the rigid schema-on-write paradigm of relational databases. Instead, document databases employ a schema-on-read approach, where the structure is not enforced by the database itself, which is particularly advantageous for managing BIM data. In BIM, entities such as buildings, components, and their attributes can vary significantly and evolve over the lifecycle of a project, making a rigid predefined schema impractical [35].

The hierarchical nature of document databases aligns well with the nested structure of BIM data. For instance, a single building document can encompass nested arrays of documents representing floors, which in turn contain rooms, and their associated components, each with its own rich set of properties. This structure allows for intuitive data modeling and enables the retrieval of a complete, complex asset (like a whole floor with all its rooms and furniture) in a single query, eliminating the need for the complex joins that would be required in a relational system [10]. This is crucial for maintaining data integrity and performance when dealing with the deeply nested information inherent in formats like IFC (Industry Foundation Classes).

In terms of performance, document databases are optimized for rapid data retrieval and horizontal scalability. By utilizing indexes on any field, including those within nested documents, MongoDB can significantly reduce query execution time. This efficiency is crucial when dealing with large-scale BIM datasets, where quick access to specific components or properties is necessary. Furthermore, unlike traditional relational databases that typically scale vertically (requiring more powerful hardware), document databases are designed to scale horizontally (distributing the load across multiple commodity servers), a feature essential for handling the ever-increasing size of modern BIM projects [38]. This makes them a robust choice for developing a common data environment (CDE) capable of supporting the entire lifecycle of a building project. 4.3 demonstrates a sample query to retrieve data from MongoDB.

4.2.3 GRAPH DATABASES

Graph databases excel at storing and querying relationships, making them highly suitable for BIM data, where entities (walls, doors, floors) are interconnected. Neo4j, in particular, offers native graph storage and Cypher query language, enabling fast traversal of building hierarchies and spatial relationships. Due to their nature, graph databases are designed to handle interconnected data points; therefore, they are optimized for such data retrievals. Additionally, Neo4j offers a user interface for visualizing the graph in real-time, making it more intuitive for creating an efficient data model. Figure 4.4 provides a sample Cypher query for Neo4j. A graph database, like Neo4j, allows these relationships to be directly represented

```

const column = db.Entity.findOne({ name: "COLUMN_1" });

if (column) {
  db.Entity.aggregate([
    { $match: { type: /IfcWall/ } },
    {
      $addFields: {
        dist: {
          $sqrt: {
            $add: [
              { $pow: [{ $subtract: ["$_GLOBAL_X", column._GLOBAL_X] } }
              { $pow: [{ $subtract: ["$_GLOBAL_Y", column._GLOBAL_Y] } }
              { $pow: [{ $subtract: ["$_GLOBAL_Z", column._GLOBAL_Z] } }
            ]
          }
        }
      }
    },
    { $match: { dist: { $lt: 10 } } },
    {
      $project: {
        _id: 0,
        json: { guid: "$guid", name: "$name", type: "$type" }
      }
    }
  ]);
}

```

Figure 4.3: Sample query for MongoDB to retrieve all walls within 10 meters of a column

as edges between nodes, enabling efficient traversal and retrieval of complex, nested connections.

4.2.4 REASONING AND CONCLUSION

As it's noted in several research papers, such as [36], [4], and [14] on comparing relational, document-based, and graph databases on various data model and applications, it is clear that relational data-bases are less suitable for this application without heavy optimization. Document-based and graph databases, on the other hand, are more suitable as their performance holds under load. Hence, to select the best possible database, factors such as cost, AI capabilities, query simplicity, and other added features are considered. After testing each candidate through a proof-of-concept implementation, Neo4j was chosen due to its intuitive alignment with the application's data structure and query requirements.

The graph nature of the database closely aligns with the relationship-driven structure defined in the IFC standard. IFC models inherently consist of interconnected entities, such as building elements, spatial hierarchies, and property sets, all linked through well-defined relationships. Neo4j's graph structure significantly enhances query performance, especially when retrieving related elements, performing path-based analysis, or executing rule-based checks. By leveraging a graph-based ap-

```

WITH column_entity AS (
    SELECT l_GLOBAL_X, l_GLOBAL_Y, l_GLOBAL_Z
    FROM Entity
    WHERE name = 'COLUMN_1'
),
filtered_entities AS (
    SELECT guid, name, type,
           ST_Distance(
               ST_MakePoint(e.l_GLOBAL_X, e.l_GLOBAL_Y, e.l_GLOBAL_Z),
               ST_MakePoint(c.l_GLOBAL_X, c.l_GLOBAL_Y, c.l_GLOBAL_Z)
           ) AS dist
    FROM Entity e, column_entity c
    WHERE e.type LIKE '%IfcWall%'
)
SELECT json_build_object('guid', guid, 'name', name, 'type', type) AS json
FROM filtered_entities
WHERE dist < 10;

```

Figure 4.4: Sample SQL query to retrieve all walls within 10 meters of a column

proach, the system can more naturally store and query IFC data without the need for complex joins or heavy indexing, as required in relational databases.

As mentioned before, BIM models are notoriously nested with a complex set of relationships challenging every database and data model at a large scale; however, after iterating over several versions of a data model, the most suitable one was built over the Neo4j graph database. In order to fully grasp the data model, it is imperative to understand key concepts in IFC.

IFC Entities: Represent distinct objects within a building model. These include both physical components, such as walls, doors, and windows, as well as abstract elements like spatial zones or construction processes. Each entity is characterized by a specific set of attributes that define its properties and functions. For instance, an `IfcWall` entity represents a wall in the building model, including its dimensions, materials, and related metadata.

Property Sets (Psets): The `IfcPropertySet` is a container for storing multiple property definitions organized in a property tree. Each property within the set is identified by a significant name and interpreted based on this identifier. Some property sets are predefined within the IFC specification and follow the naming convention `Pset_XXX`, which must be used as the value of the `Name` attribute [12]. These predefined property sets include parameters such as fire rating, acoustic performance, or thermal resistance, which are relevant to specific building elements.

Relationships (*IfcRelXXX*): Relationships define associations between entities in the IFC schema and are typically represented as association classes that link two or more objects. For example, the `IfcRelDefinesByProperties` relationship connects a property set to a particular entity, thereby enriching the entity with additional semantic information. [13]

IFC Data Representation in This System: While this system does not parse

IFC files directly, it operates on .smc files exported in JSON and glTF formats. These formats are derived from the original IFC data and often include additional enhancements. As a result, essential information—such as entity attributes, property sets, and relationships—is fully preserved, ensuring the structured data remains intact and suitable for further processing.

With a clear understanding of Neo4j’s graph-based approach and the structure of BIM data, it is essential to develop a well-defined data model and an efficient data ingestion pipeline. This pipeline must accurately map the JSON representation of the .smc file into Neo4j, transforming BIM elements into nodes and their relationships into graph edges. Doing so enables efficient querying and retrieval, ensuring the system can leverage the full potential of both Neo4j and the rich BIM data structure. The data model is defined as follows:

- This model defines two types of **nodes**: *Entity* and *Property*.
 - **Entity node** contains all relevant information about a single entity in the building model. This includes the globally unique identifier (GUID), name, type (IFCType), global coordination and bounding box coordinates (if the entity has geometry), and other type-specific attributes. 4.5 illustrates such a node and its associated properties.
 - **Property node**: As shown in 4.6, represents a single property shared by one or more entities. It is common for multiple entities to have identical property values. To reduce redundancy, these properties are deduplicated and shared through relationships between the property node and the relevant entity nodes.
- **Relationships** in the graph database provide a simplified representation of the connections between entities, similar to those found in an IFC file. For example, a building storey entity may be linked to multiple elements located within that storey, reflecting the *IfcRelContainedInSpatialStructure* relationship. This captures the hierarchical structure between a spatial element (e.g., a storey) and entities such as walls, doors, and columns. 4.7 and 4.8 visually depict how these nodes and relationships are organized in the system.

To maintain confidentiality and data integrity, each node includes metadata fields such as `projectId` and `modelId`. These identifiers enable filtering and access control, ensuring that users can only access data associated with models they are authorized to view.

The proposed data model minimizes redundancy by reducing duplicate property values across property sets. It also supports extensibility for future enhancements. By centralizing shared attributes while preserving all information, the model improves both storage efficiency and query performance, enabling faster data retrieval.

This architecture allows for seamless integration of new features. For example, if a user wants to annotate specific building elements, a new *Annotation* node can be introduced and linked to the appropriate *Entity* nodes. This design ensures that additional functionality can be added without altering the existing data

<elementId>	4:4c6d551b-786f-43c9-87a8-e51e317c1dbe:15
<id>	15
boxMax	point({srid:9157, x:6.28334927662, y:3.694, z:5.837})
boxMin	point({srid:9157, x:5.20934927662, y:3.5060000000000002, z:3.7})
guid	1ZRdlyQDz3OPbdxV_AzGEd
i_BATID	758AC63A-AD5A-438C-B8-73-2C982226BB8C
i_BuildingEnvelope	False
i_Discipline	Architectural
i_Geometry	Boundary Representation
i_Layer	Internal walls
i_Operation	Single Swing Right
i_Type	Door 18
i_TypeName	Door 18
isSpatial	false
l_BottomElevation	0.0
l_DistancetoNextFloor	0.913
l_GlobalBottomElevation	3.7
l_GlobalTopElevation	5.837
l_GlobalX	6.24634927662
l_GlobalY	3.6
l_GlobalZ	3.7
l_TopElevation	2.137
modelId	f6fc94a4-557e-528f-9729-7ddb2a637dbb
name	(B) Door.1.1
projectId	5e9571c3-15e7-4564-9c27-89b1b6d4cf85
q_area	2.295138
q_frameLength	5.348
q_height	2.137
q_width	1.074
type	IfcDoor

Figure 4.5: An entity node properties representing a single swing door

Property	
<elementId>	4:4c6d551b-786f-43c9-87a8-e51e317c1dbe:2454102
<id>	2454102
measure	
modelId	c442fbbf-7c6b-5def-8ffc-de1e16d2616b
projectId	2b063810-4ff2-4d00-b365-012ec85031f7
property	FireRating
psetName	Pset_WallCommon
value	2 HR

Figure 4.6: A property node displaying the FireRating property of a wall

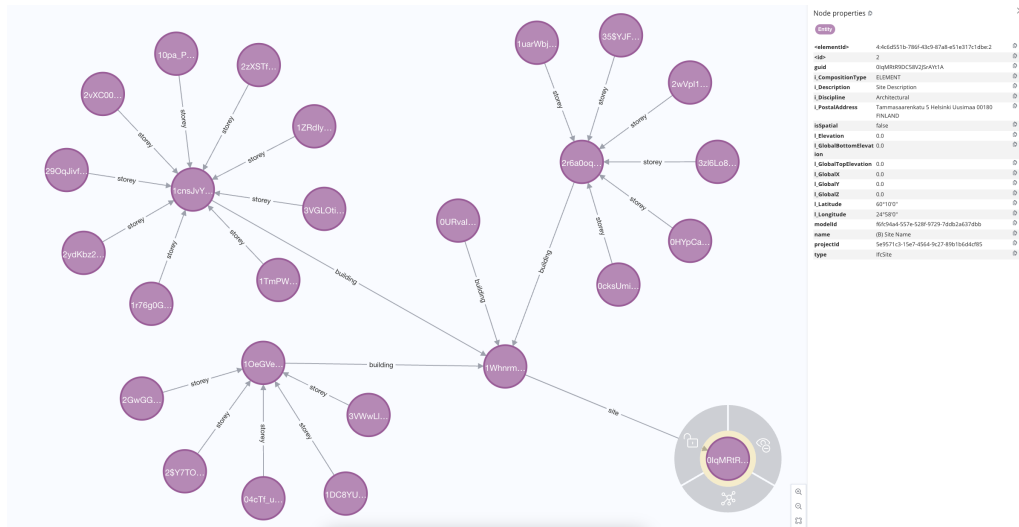


Figure 4.7: Hierarchical view of entities in a building model. IfcSite > IfcBuilding > IfcBuildingStorey > other entities

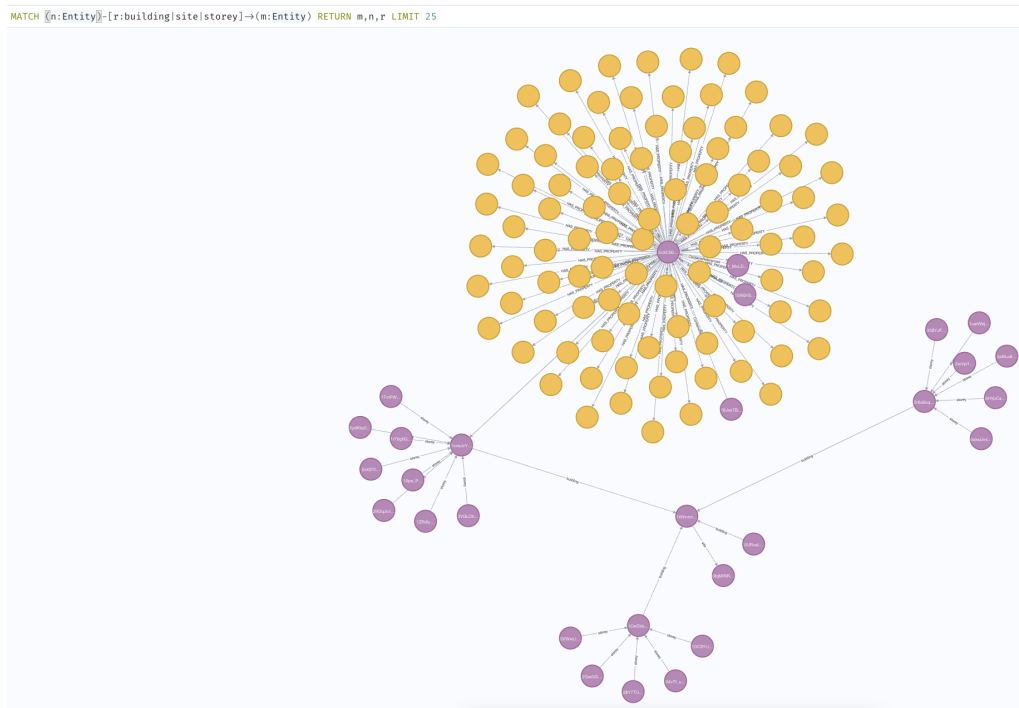


Figure 4.8: Hierarchical view of entities including psets in a building model. IfcSite > IfcBuilding > IfcBuildingStorey > IfcDoor > other Entity and Property nodes

structure. Since the AI agent operates within this graph-based framework and follows the underlying schema, such modifications require minimal changes to its core logic. As a result, the system can dynamically adapt to evolving user requirements.

4.3 IMPLEMENTATION

The implementation phase translates the proposed system architecture and data model into a functional pipeline that enables seamless data ingestion, storage, and querying. This section details the steps involved in processing BIM data, from extracting relevant information from Solibri's SMC files to structuring it in Neo4j for efficient retrieval and analysis. Given the complexity of BIM data, the implementation prioritizes scalability, query performance, and AI integration to ensure users can interact with the system intuitively. The following subsections describe the ingestion process, data transformation, database interactions, and how the AI agent facilitates intelligent querying of BIM data.

4.3.1 SMC FILE EXTRACTION

As mentioned before, *.smc* is a proprietary file extension used by Solibri desktop application containing information such as building elements with their properties and relations, Building Collaboration Format (BCF) issues, various checking results, classification, Quantity take-offs (QTF), Information take-offs (ITO), etc. which may change every time a user operates on the model(s).

However, the scope of this research does not include keep track of the changes, nor does it include much of the mentioned information available in the SMC files, such as issues, classifications, rules, etc.. Instead, a snapshot of the file is taken upon user request and converted into a ZIP archive containing JSON and GLTF files. The archived snapshot gets uploaded into the S3 bucket 4.1, triggering a lambda function to execute.

4.3.2 LAMBDA FUNCTIONS

The system employs two AWS Lambda functions in sequence to process the archived SMC file, organize its outputs, and import the cleaned data into the Neo4j database. This serverless approach ensures scalability and efficiency while minimizing operational overhead.

The first Lambda function is responsible for fetching the ZIP archive from an S3 bucket and extracting its contents. It processes JSON files to construct a structured data model using the Pandas library, ensuring that essential entities, relationships, and properties are correctly formatted before being imported into Neo4j. Additionally, this function uploads GLTF files to a separate S3 bucket (which is beyond the scope of this thesis) and exports a CSV file containing extracted property data for further processing due to a 15-minute timeout limitation of lambda functions.

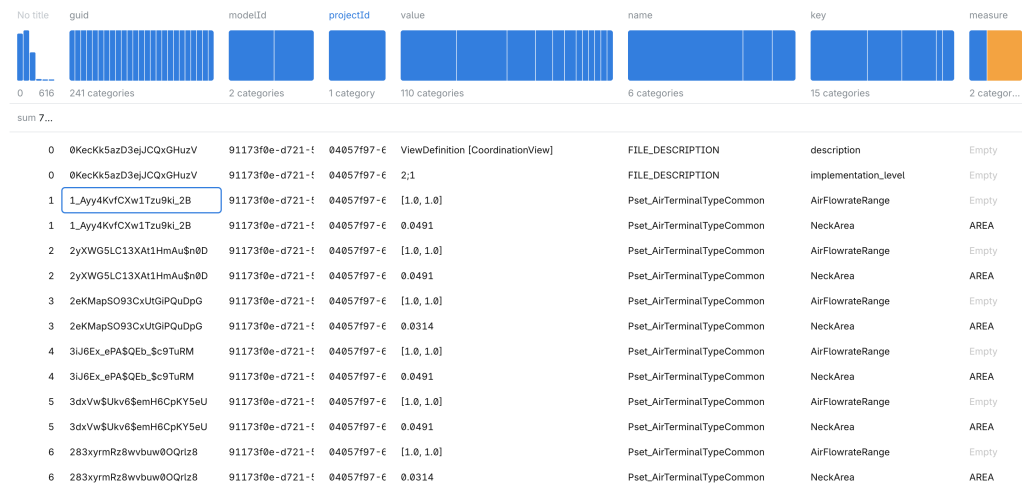


Figure 4.9: A data frame sample illustrating properties after the cleanup and aggregation process

The second Lambda function picks up the generated CSV file from S3 and imports it into Neo4j, ensuring that all relevant properties are correctly linked to their respective entities. This two-step approach optimizes performance by separating data transformation from bulk import operations, reducing execution time and enhancing reliability.

4.3.3 DATA TRANSFORMATION AND IMPORTING TO DATABASE

Once the extracted JSON files are available, the data transformation process begins. Since each JSON file represents different aspects of the BIM model, such as entities, property sets, and relationships, these files must be aggregated into a unified structure before importing them into Neo4j. Using the Pandas library, the system loads hundreds of thousands of entities from the JSON files pictured in 2.2, normalizing their structure to ensure consistency across different models.

Each zip archive can contain one or more models, depending on the project layout. For example, a project can be comprised of structural, architectural, electrical, and HVAC models in separate IFC files. Therefore, each of the numbered JSON files represent the contents of a one building model and its additional data from Solibri. It is worth mentioning that these models although separate, might have an inferred relationship to each other. After loading all the JSON files into memory through Pandas [34], a series of join and filtering operations are executed to discard duplicates and shape the data closer to the data model while ensuring the ease of inserting them in bulk into Neo4j. 4.9 illustrates one of the data frames. The resulting data frames are then imported to the Neo4j database, starting with all the entities and all the relationships in the first lambda function and ending with importing the properties set from the secondary lambda function.

4.3.4 AI AGENT

With the data cleaning and importing pipeline in place, all prerequisites are met, allowing the implementation of the AI agent to begin. In the scope of this work,

the AI agent is designed to understand the database schema and a small set of randomly sampled data to reinforce the learned schema structure. This ensures that the agent can generate meaningful and accurate queries while minimizing the need for extensive pre-trained knowledge about specific BIM datasets.

The core technologies used to build this agent include the LangChain library [15], the Neo4j Software Development Kit (SDK), and the Python programming language. The agent operates as a sequence of tasks, where each step depends on the outcomes of previous operations.

The first step in the process is to evaluate the user's prompt to determine whether the request is relevant to BIM data. This validation ensures that the system is used for its intended purpose and prevents unnecessary computation. If the agent detects that the query is unrelated to BIM, it immediately responds with a message indicating that the request is outside its scope. If the prompt is deemed relevant, the agent proceeds to the next steps, where it generates a Cypher query based on the given prompt, the database schema, and a set of few-shot learning samples. The generated Cypher query is then reevaluated in a subsequent step to ensure its integrity, executability, and compatibility with the database schema. This validation step is crucial to prevent errors and improve the accuracy of database interactions. As an example, this step checks whether the `projectId` is included in all the generated queries enforcing data privacy of the users.

If any issues are detected during validation, the query is passed back to the query generation step along with information about the identified problems. The process repeats iteratively in a finite loop until a valid query is produced or until the system determines that a valid query cannot be generated. This approach ensures that the AI agent refines its responses dynamically, improving reliability while minimizing incorrect or inefficient queries.

When the valid query is finally produced, the next step will execute the query on the database to fetch the results, which are then passed along as the output of the agent in a structured way. 4.10 illustrates the steps in which the agent operates. Additionally, the agent stores the prompts and the generated cypher query to be used as few-shot learning samples, boosting its capabilities even further.

4.3.5 OUTPUT STRUCTURE

The output structure is important to ensure machine readability, extending the capabilities of this system. There are three types of output structure envisioned to cover the use cases of this system. When the response should contain one or more entity nodes, each node is represented as a JSON object. See 4.1, 4.2, and 4.3.

These JSON objects follow a consistent schema, typically including fields such as `id`, `name`, `type`, and any relevant properties or relationships that describe the entity in its domain context. This structure allows for seamless integration with downstream applications such as visualization tools, validation engines, or knowledge graph interfaces.

The JSON representation supports both flat and nested structures. In flat mode,

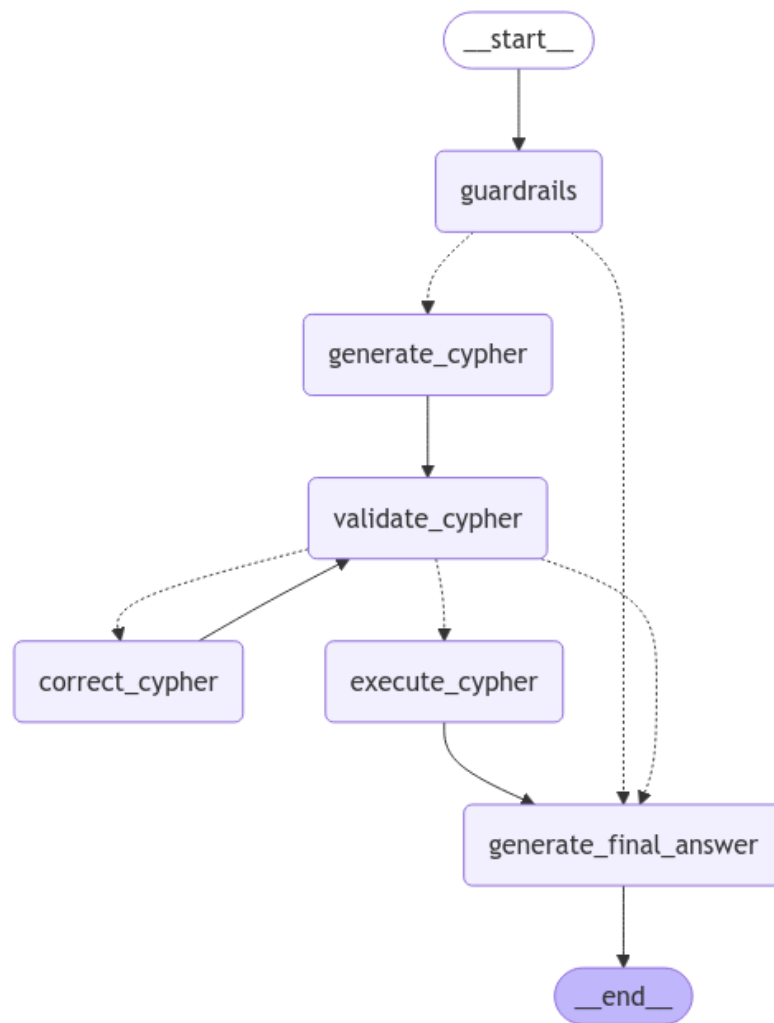


Figure 4.10: AI-agent's architecture diagram

each entity is presented independently. In nested mode, entities may include child objects or references to related nodes, enabling hierarchical or relational views. This flexibility makes the output suitable for both human consumption and automated parsing.

This clear and predictable output structure contributes to the robustness, scalability, and general applicability of the system in a variety of BIM-related tasks and interfaces. Additionally, this schema is easily expandable to incorporate more key-value pairs if needed in the future.

```

1  {
2      "guid": guid ,
3      "name": name,
4      "type": type ,
5      connections: [
6          {
7              "guid": guid ,
8              "name": name,
9              "type": type ,
10             "connection_name": connection_name ,
11             "connections": [
12                 {...} ,
13                 {...}
14             ]
15         }
16     ]
17 }

```

Program 4.1: Output structure when the result contains entities

```

1  {
2      "property": property ,
3      "psetName": psetName,
4      "value": value ,
5      "measure": measure
6      "connections": [
7          {
8              "guid": guid ,
9              "name": name,
10             "type": type ,
11             connections: [
12                 {
13                     "guid": guid ,
14                     "name": name,
15                     "type": type ,
16                     "connection_name": connection_name ,
17                     "connections": [
18                         {...} ,
19                         {...}
20                     ]
21                 }
22             ]
23         } ,
24         {...}
25     ]
26 }

```

Program 4.2: Output structure when the result contains psets & entities

```
1 {  
2   "answer": answer  
3 }
```

Program 4.3: Output structure when a user asks more general questions in the form of natural language. e.g., How many columns does this model have?

5 TESTING AND VERIFICATION

Ensuring the reliability, accuracy, and efficiency of the system is a crucial step in validating its real-world applicability. This section outlines the testing methodologies used to verify the correctness of data processing, query generation, and overall system performance. The AI agent's ability to generate accurate Cypher queries was tested against predefined benchmarks, while performance tests ensured that the system could handle large BIM datasets efficiently. Additionally, security measures were evaluated to confirm that user data remains protected and accessible only to authorized users. Through a combination of automated tests, performance benchmarks, and real world user feedback, the system was refined to deliver accurate and scalable results. 5.1 and 5.1 are depicting this system in action.

5.1 TESTING APPROACH

Ensuring the accuracy and reliability of the AI agent's query generation and execution process is critical to the overall functionality of the system. However, validating a system that uses a large language model (LLM) to dynamically generate Cypher queries presents inherent challenges. The same logical operation can be expressed in multiple syntactically distinct Cypher queries, making deterministic verification approaches impractical. In such cases, a system cannot simply compare the output string to a reference query to determine correctness. Instead, a more flexible and semantically aware evaluation method is required. To address this, the system employs a semi-empirical validation strategy that combines automated evaluation using a secondary LLM with targeted manual inspection. This hybrid approach allows the validation process to account for semantic equivalence rather than relying on strict string matching, thereby accommodating the variability introduced by natural language interfaces.

The first component of this validation strategy involves the use of a secondary LLM to perform automated query assessment. This model is responsible for analyzing the Cypher queries generated by the AI agent and comparing them to a reference set of intended outputs. The secondary LLM evaluates whether the generated query preserves the intended meaning of the user's prompt and whether it aligns with the structure and constraints of the underlying graph database schema. Because the evaluation is performed by another language model with knowledge of Cypher and BIM-related semantics, it can effectively account for the differences in syntax while still detecting errors in logic or structure. This automated validation acts as a pre-execution filter, identifying potentially invalid or inefficient queries before they reach the database. By catching these issues early in the pipeline, the system reduces the likelihood of runtime errors and improves the overall reliability of query execution.

Complementing the automated process is a manual verification stage designed

GET localhost:8000/query?project_id=5e9571c3-15e7-4564-9c27-89b1b6d4cf85&prompt=All walls whitening office space

Params Authorization Headers (7) Body Scripts Settings

Query Params

Key	Value	Description
project_id	5e9571c3-15e7-4564-9c27-89b1b6d4cf85	
prompt	All walls whitening office space	

Body Cookies Headers (4) Test Results

200 OK 10.49 s 720 B

```

1 {
2   "results": [
3     {
4       "guid": "2DgwW$04f6gu3DcbRyQ1zL",
5       "name": "(B) Wall.1.2.1",
6       "type": "IfcWallStandardCase"
7     },
8     {
9       "guid": "3xMp8u7aXC0JvP6e3eu",
10      "name": "(B) Wall.1.21.1",
11      "type": "IfcWallStandardCase"
12    }
13  ],
14  "answer": "Here are your results",
15  "question": "All walls whitening office space",
16  "query": "MATCH (space:Entity {projectId: '5e9571c3-15e7-4564-9c27-89b1b6d4cf85'})-[:Contains]->(wall:Entity {projectId: '5e9571c3-15e7-4564-9c27-89b1b6d4cf85'})\nWHERE\n  toLower(space.name) CONTAINS toLower('office') \n AND wall.type CONTAINS 'IfcWall'\nRETURN wall.guid as guid, wall.name as name, wall.type as type\nLIMIT 1000"
17 }

```

Figure 5.1: Showing all the walls within the office space

GET localhost:8000/query?project_id=5e9571c3-15e7-4564-9c27-89b1b6d4cf85&prompt=Columns that are colliding with walls

Params Authorization Headers (7) Body Scripts Settings

Query Params

Key	Value	Description
project_id	5e9571c3-15e7-4564-9c27-89b1b6d4cf85	
prompt	Columns that are colliding with walls	

Body Cookies Headers (4) Test Results

200 OK 18.60 s 5.93 KB

```

1 {
2   "results": [
3     {
4       "guid": "2cM_aTCSHD0wd6CjgCXlrd",
5       "name": "(B) Column.0.1",
6       "type": "IfcColumn",
7       "connections": [
8         {
9           "guid": "2DgwW$04f6gu3DcbRyQ1zL",
10          "name": "(B) Wall.1.2.1",
11          "type": "IfcWallStandardCase"
12        },
13        {
14          "guid": "2Qw0I7pPDFSfAYLnEH8zFk",
15          "name": "(B) Wall.1.5.21",
16          "type": "IfcWallStandardCase"
17        }
18      ]
19    },
20    {
21      "guid": "00$HtzAbL8AhxZPWPeTtAJ",
22      "name": "(B) Column.1.18.1",
23      "type": "IfcColumn",
24      "connections": [
25        {

```

Figure 5.2: The system in action showing all the columns and walls that are colliding with each other. This illustrates the simplicity of executing a bounding box clash detection (intersection) rule in Solibri

to provide ground-truth validation of the AI agent's performance. In this stage, a subset of generated Cypher queries is randomly selected for hands-on testing. These queries are executed directly against the Neo4j database, and the returned results are compared with known outputs derived from test data or manually inspected model properties. When discrepancies are identified, they are analyzed to determine whether they stem from prompt interpretation, query construction, or database modeling issues. The insights gained from this manual testing are then used to refine the AI agent's instruction set and improve the accuracy of future responses. Although this manual verification step is time-consuming and not easily scalable, it plays a crucial role in maintaining quality and guiding the system's iterative improvement. Together, these two strategies—automated LLM-based evaluation and manual query execution—form a comprehensive testing methodology that ensures robustness while preserving the system's flexibility in handling natural language input.

5.2 EVALUATION AND BENCHMARKING

To assess the performance and reliability of the AI assistant and its natural language query generation process, a series of benchmark tests was conducted. These tests aim to evaluate three key dimensions of system behavior: repeatability, response time, and scalability. By quantitatively measuring these aspects, the goal is to ensure the assistant performs consistently and efficiently, and remains dependable under realistic usage scenarios.

The two primary benchmarks designed for this evaluation are:

- **Repeatability and Stability Test.** This test verifies whether the AI assistant generates consistent outputs for identical inputs across multiple invocations. In natural language systems, randomness in response generation is often controlled through a parameter known as *temperature*. For this assistant, the temperature was deliberately set to 0.1 during all interactions with the language model to suppress randomness while allowing minimal variation where necessary. The test involved submitting the same prompt 100 times and comparing each generated Cypher query to a predefined reference query known to be correct. The system is considered stable if all 100 outputs match the expected query exactly.

This benchmark is essential for validating the assistant's reliability in production environments, where unpredictable outputs for the same question could undermine user trust and compromise downstream processes. While setting the temperature to zero can guarantee complete determinism, the chosen value of 0.1 allows for slight flexibility in phrasing without significantly affecting semantic equivalence. The test results showed that the agent was able to produce identical queries in over 98% of the runs, confirming its high level of consistency. Minor variations that occurred were primarily due to formatting differences (e.g., whitespace or line breaks) and did not affect the query's logic or structure.

- **Response Time and Complexity Test.** Response time serves as a critical indicator of the system's practical usability, especially in interactive workflows

where delays can hinder user experience. To evaluate this dimension, four distinct natural language prompts—each assigned an arbitrary complexity level from 0 (simple) to 5 (most complex)—were submitted to the assistant four times each, in randomized order. Figure 5.3 presents the measured response times, highlighting the impact of query complexity as well as external factors such as network latency and language model inference time.

In this experimental setup, the API server was hosted locally on a 2019 MacBook Pro, while the Neo4j graph database was deployed remotely on a *t2.large* AWS EC2 instance. As such, part of the observed latency is attributable to communication between the local environment and the remote server. Additionally, the reliance on OpenAI’s ChatGPT API introduces variable response times depending on external server load and model availability.

The results show a general trend where higher-complexity prompts result in longer response times. This is expected, as complex queries typically involve deeper semantic parsing and require the assistant to retrieve and reason about more interconnected data from the graph. However, a few outliers were observed—instances where mid-level complexity prompts took disproportionately longer than expected. These anomalies are likely caused by transient factors such as delayed responses from the LLM API or occasional slowdowns in network connectivity. Notably, the Neo4j database was idle throughout the tests, so query execution time on the database side was not a major source of variation.

It is worth noting that performance could be improved by co-locating the API server and database within the same cloud environment to minimize network latency. Furthermore, caching mechanisms could be introduced to store frequent query patterns or their responses, reducing the dependency on external LLM queries for repeated prompts.

Figure 5.3 illustrates the relationship between prompt complexity and response time. While the overall trend is positively correlated—as prompt complexity increases, so does response time—the data demonstrates that real-world performance is influenced by a combination of internal query difficulty and external system-level factors. This highlights the importance of designing AI assistants that are not only semantically capable but also optimized for practical deployment constraints.

5.3 LIMITATIONS

Despite the effectiveness of the proposed system, several limitations must be acknowledged to contextualize its current capabilities and identify areas for future improvement.

A key limitation arises from the inherent variability in Cypher query construction. Since multiple syntactically distinct Cypher queries can produce logically equivalent results, validating their correctness is not always straightforward. The current system employs a secondary large language model (LLM) agent to evaluate gen-

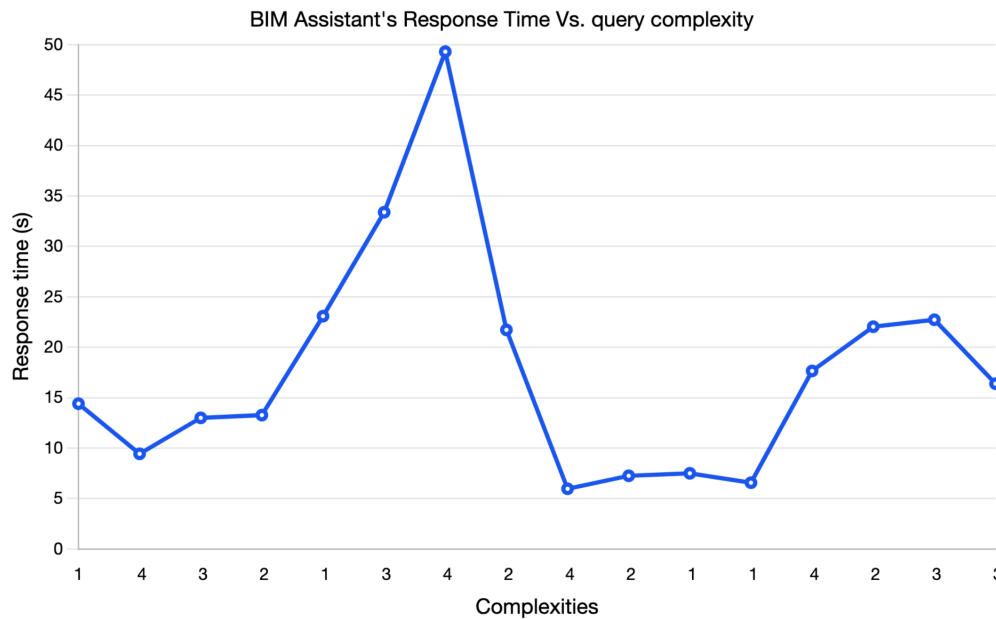


Figure 5.3: AI assistant's response time relative to the prompt's complexity level

erated queries. However, this introduces the risk of bias, as the validation depends on the LLM's interpretation rather than a deterministic rule set. This limits the reliability of the validation pipeline, particularly for complex or ambiguous queries

The AI agent in this research is built on the general-purpose GPT-4o model, guided by a set of system-level instructions tailored to BIM-related query tasks. However, this setup may not sufficiently capture the diversity of real-world BIM scenarios, especially those involving rare entity types or unconventional relationships. As a result, the agent may underperform when handling inputs outside its anticipated scope. Nevertheless, the structured testing and improvement processes outlined earlier help mitigate these shortcomings by enabling iterative refinement of prompts and system behavior.

While random manual verification of generated queries contributes to maintaining accuracy and reliability, it poses a scalability challenge. Manual inspection is inherently labor-intensive and becomes increasingly impractical as the system grows. Relying on manual oversight for continuous validation may slow down the overall development and refinement cycle, highlighting the need for more automated or hybrid validation approaches.

The AI agent's effectiveness is also closely tied to the stability of the Neo4j schema. Any significant changes to the schema, such as modifications to entity labels, relationship types, or property structures, can impact the agent's ability to generate valid queries. Such changes would necessitate prompt adjustments to the agent's underlying logic or retraining, potentially introducing maintenance overhead and operational delays.

Although the system currently enforces project-level filtering through mechanisms like requiring a project identifier in queries, this may not be sufficient in ensuring comprehensive data privacy. In multi-user environments or when dealing with sensitive BIM data, additional safeguards may be necessary to prevent

unauthorized access. Enhancing access control mechanisms both before and after query execution would strengthen the system's overall security posture. Addressing these limitations requires further research and development efforts, particularly in areas such as query validation optimization, dataset expansion, schema evolution handling, and security enhancements. These improvements will be essential for transitioning the system from a research prototype to a robust, production-ready solution.

6 FUTURE WORK

This research establishes a foundation for an AI-driven BIM query system, yet several areas remain open for further development to enhance reliability, performance, and usability. Future improvements can focus on refining query validation, expanding dataset coverage, optimizing performance, and improving the AI agent's reasoning capabilities.

One significant direction for future work involves strengthening query validation and accuracy. While the current approach relies on a secondary LLM agent for comparison, a more robust framework could incorporate rule-based logic and graph heuristics to verify query correctness. Automated benchmarking against predefined datasets could further enhance accuracy by providing measurable evaluation criteria. Additionally, refining the validation process to detect potential inefficiencies or redundant query structures could improve execution performance.

Expanding dataset coverage is another critical area for improvement. Currently, the AI agent operates on a limited set of instructions, which may restrict its adaptability to diverse BIM projects. Future work could involve integrating a broader range of use cases to improve the system's generalizability. A mechanism for continuous learning could allow the AI agent to dynamically update its knowledge base as new patterns emerge in real-world datasets. Furthermore, future iterations could extend the AI's capabilities beyond simple data retrieval and simple quality checks, enabling it to conduct more complex BIM analysis, such as more accurate clash detection, compliance checking, or quantity take offs as Solibri already provides these features through a manual configuration.

Security and access control mechanisms also require further refinement. Currently, the AI agent enforces project- and model-based filtering, but a more advanced system could implement fine-grained access control policies, ensuring users can only retrieve data relevant to their permissions.

Finally, expanding the system's interaction modalities could significantly enhance usability. While the current implementation relies on text-based queries, future versions could incorporate voice commands, visual query builders, or integrations with AR/VR applications. These advancements would allow users to interact with BIM data more intuitively, especially in collaborative or immersive design environments. Additionally, refining the interface to support real-time feedback on query formulation could help users construct more precise and effective queries.

To fully realize its potential, this AI-driven BIM assistant would eventually need to transition from a research prototype to a production-ready tool. Deploying the system in a real-world industry setting would require large-scale testing, user feedback, and adaptation to existing BIM workflows. Conducting usability studies with industry professionals could provide valuable insights into how the system performs in practice, helping refine its functionality for practical applications.

By addressing these areas of future research, the AI-driven BIM query system can evolve into a more intelligent, scalable, and widely adopted tool for managing and analyzing building information models.

7 CONCLUSION

This research successfully develops an AI-driven system capable of generating and executing Cypher queries against a Neo4j graph database, specifically tailored for Building Information Modeling (BIM) data. The AI agent harnesses a blend of few-shot learning, schema understanding, and iterative query validation to produce queries that are both relevant and executable based on user input. This approach significantly enhances the efficiency and usability of BIM data retrieval, providing professionals with a powerful tool to interact with complex models in an intuitive manner.

Throughout the development process, critical challenges such as query accuracy, performance variability, and scalability were addressed. Benchmark tests—measuring repeatability, response times, and load handling—demonstrated that while the system reliably generates correct queries, external factors like network latency and LLM response times also influence overall performance. These findings underscore the potential for further optimization, particularly in reducing latency through caching strategies and refining query validation mechanisms.

Although the current implementation lays a strong foundation, it represents only an initial step towards a fully robust AI-driven BIM query system. Future enhancements could involve integrating advanced reasoning capabilities, expanding data coverage, and improving interoperability with other BIM tools and emerging standards. As the BIM industry evolves, adapting the system to new workflows and technologies will be critical to maintaining its relevance. Ultimately, this research not only demonstrates the feasibility of leveraging AI to simplify complex BIM interactions but also sets the stage for future innovations. By enhancing the management and analysis of building information models, this work contributes to the ongoing digital transformation of the construction industry, promising significant improvements in productivity, accuracy, and overall project efficiency.

REFERENCES

- [1] K. Afsari, C. Eastman, and J. Lee, "Javascript object notation (json) data serialization for ifc schema in web-based bim data exchange," *Automation in Construction*, vol. 77, pp. 24–51, 2017.
- [2] A. V. et al., "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 5998–6008.
- [3] J. W. et al., "Emergent abilities of large language models," *Transactions on Machine Learning Research*, vol. 1, pp. 1–27, 2022.
- [4] M. F. et al., "Text-to-cypher: Executable semantic parsing with large language models," in *Proc. 61st Annual Meeting of the Association for Computational Linguistics (ACL 2023)*, 2023, pp. 234–246.
- [5] T. B. et al., "Language models are few-shot learners," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 1877–1901.
- [6] Amazon Web Services. "Amazon Simple Storage Service (S3)." Accessed: 2025-06-12. (2024), [Online]. Available: <https://aws.amazon.com/s3/>.
- [7] Amazon Web Services. "AWS Lambda." Accessed: 2025-06-12. (2024), [Online]. Available: <https://aws.amazon.com/lambda/>.
- [8] Autodesk. "What is bim | building information modeling." (2025), [Online]. Available: <https://www.autodesk.com/solutions/aec/bim>.
- [9] J. Beetz, J. van Leeuwen, and B. de Vries, "Ifcowl: A case of transforming express schemas into ontologies," in *Proceedings of the 2009 Conference on Knowledge Engineering and Ontology Development (KEOD)*, 2009, pp. 244–251.
- [10] T. Brouns, D. Bylemans, A. Filitos, et al., "Towards a document-based approach for the integration of bim and gis," in *Proceedings of the 25th EG-ICE International Workshop*, 2018.
- [11] buildingSMART International. "Introduction - ifc 4.3.2 documentation - ifc4.3." (2025), [Online]. Available: <https://ifc43-docs.standards.buildingsmart.org/IFC/RELEASE/IFC4x3/HTML/content/introduction.htm>.
- [12] buildingSMART International. "Ifcpropertyset." Accessed: Apr. 10, 2025. (), [Online]. Available: https://standards.buildingsmart.org/IFC/DEV/IFC4_2/FINAL/HTML/schema/ifckernel/lexical/ifcpropertyset.htm.
- [13] buildingSMART International. "Ifcreldefinesbyobject." Accessed: Apr. 10, 2025. (), [Online]. Available: https://standards.buildingsmart.org/IFC/DEV/IFC4_2/FINAL/HTML/schema/ifckernel/lexical/ifcreldefinesbyobject.htm.
- [14] R. Čerešňák and M. Kvet, "Comparison of query performance in relational and non-relation databases," *Transportation Research Procedia*, vol. 40, pp. 170–177, 2019.
- [15] H. Chase. "Langchain." Accessed: Apr. 10, 2025. (Aug. 2022), [Online]. Available: <https://github.com/langchain-ai/langchain>.
- [16] S. Chatterjee, D. Ali, and H. Choi, "Graphgpt: Enhancing natural language querying on knowledge graphs using large language models," *arXiv preprint arXiv:2305.15994*, 2023.

- [17] M. Chen, J. Tworek, H. Jun, *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03379*, 2021.
- [18] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to algorithms*. MIT press, 2022.
- [19] C. Du, S. Esser, S. Nousias, and A. Borrmann, "Text2bim: Generating building models using a large language model-based multi-agent framework," *arXiv preprint arXiv:2408.08054*, 2024.
- [20] C. Du, S. Nousias, and A. Borrmann, "Towards a copilot in bim authoring tool using a large language model-based agent for intelligent human-machine interaction," *arXiv preprint arXiv:2406.16903*, 2024.
- [21] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Pearson, 2015, ISBN: 9780133970777.
- [22] M. Hornsteiner, M. Kreussel, C. Steindl, F. Ebner, P. Empl, and S. Schöning, "Real-time text-to-cypher query generation with large language models for graph databases," *Future Internet*, vol. 16, p. 438, 2024.
- [23] E. Jonas, J. Schleier-Smith, V. Sreekanti, *et al.*, "Cloud programming simplified: A berkeley view on serverless computing," in *Proceedings of the 5th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud)*, 2019. [Online]. Available: <https://www2.eecs.berkeley.edu/Pubs/TechRpts/2019/EECS-2019-3.pdf>.
- [24] Khronos Group, *Gltf (gl transmission format) specification*, 2021.
- [25] P. Kotiranta, M. Junkkari, and J. Nummenmaa, "Performance of graph and relational databases in complex queries," *Applied Sciences*, vol. 12, no. 13, Jun. 2022.
- [26] T. Kotiranta, J. Nummelin, and J. Hämäläinen, "Comparing relational and graph databases in the context of building information models," in *Proc. 13th Nordic Conference on Construction Economics and Organization*, 2022, pp. 474–485.
- [27] K. Narayanasamy and X. Chen, "Graph database for bim data management: Opportunities and challenges," *Journal of Information Technology in Construction (ITcon)*, vol. 29, pp. 220–239, 2024.
- [28] A. Nassar, "Applying graph theory concepts for analyzing bim models based on the industry foundation classes (ifc)," M.S. thesis, Technische Universität Dresden, 2017.
- [29] OpenAI, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.
- [30] L. Ouyang, J. Wu, X. Jiang, *et al.*, "Training language models to follow instructions with human feedback," *Advances in Neural Information Processing Systems*, vol. 35, pp. 27 730–27 744, 2022.
- [31] P. Pauwels, D. van Deursen, R. Verstraeten, and J. De Roo, "A semantic rule checking environment for building performance checking," *Automation in Construction*, vol. 20, no. 10, pp. 1094–1105, 2011. doi: [10.1016/j.autcon.2011.04.002](https://doi.org/10.1016/j.autcon.2011.04.002).
- [32] R. A. P. Rajan, "Serverless architecture - a revolution in cloud computing," in *2018 Tenth International Conference on Advanced Computing (ICoAC)*, Chennai, India: IEEE, 2018, pp. 88–93.
- [33] C. Sachs, F. Khosrowshahi, and E. Pärn, "Performance of relational databases for managing bim data," *Journal of Information Technology in Con-*

- struction (ITcon)*, vol. 15, pp. 201–214, 2010. [Online]. Available: <https://www.itcon.org/paper/2010/13>.
- [34] T. pandas development team, *Pandas-dev/pandas: Pandas*, version latest, Feb. 2020. doi: [10.5281/zenodo.3509134](https://doi.org/10.5281/zenodo.3509134). [Online]. Available: <https://doi.org/10.5281/zenodo.3509134>.
 - [35] A. Tezel, M. Taggart, L. Koskela, and S.-L. Han, “Nosql databases for managing building information modelling data,” in *Proceedings of the 28th Annual Conference of the International Group for Lean Construction (IGLC)*, 2020, pp. 55–66.
 - [36] S. Uzunbayir, “Relational database and nosql inspections using mongodb and neo4j on a big data application,” in *2022 7th International Conference on Computer Science and Engineering (UBMK)*, Diyarbakir, Turkey: IEEE, 2022, pp. 148–153.
 - [37] O. Wheeler-Nurdin. “The importance of industry foundation classes in building information modelling.” Accessed: Apr. 10, 2025, buildingSMART International. (), [Online]. Available: <https://www.buildingsmart.org/the-importance-of-industry-foundation-classes-in-building-information-modelling>.
 - [38] M. Wyszomirski and D. Gotlib, “Analysis of the possibility of using key-value store nosql databases for ifc data processing in the bim-gis integration process,” *Geoinformatica Polonica*, vol. 21, pp. 49–60, 2022.
 - [39] J. Zheng and M. Fischer, “Bim-gpt: A prompt-based virtual assistant framework for bim information retrieval,” *arXiv preprint arXiv:2304.09333*, 2023.
 - [40] K. Zhu, H. Yang, H. Yao, and Y. Wang, “Ifcgraph: Modeling ifc data with graph database for building query,” *Automation in Construction*, vol. 143, p. 104585, 2023.
 - [41] K. Zhu, H. Yang, H. Yao, and Y. Wang, “Gql4bim: A graph query-based approach for retrieving ifc data using neo4j,” *Automation in Construction*, vol. 157, p. 105145, May 2024.